NVIS Rack Status

16K User and Deployment Guide

PythonCM-primary release edition · Version 3.0

NVIDIA GB300 NVL72 · NscaleTX

> **Start here.** This guide assumes you have never used Rack Status, BCM, PythonCM, or the NscaleTX validation workflow. It explains what every major screen means, which actions are safe, and how to preserve evidence before changing node state.

Guide date: 14 September 2026

Application: NVIS Rack Status / PythonCM-primary 16K deployment

Prepared by: Harish Lakshminarasimhan

Contents

About this guide 5
What changed in this release 5
Five rules to remember 5
1. Your first ten minutes 6
1.1 Open and authenticate 6
1.2 Read the Overview page 6
1.3 Know where things live 7
2. The 16K cluster mental model 7
2.1 SU, rack, tray, node, and GPU 7
2.2 Inventory facts versus campaign denominators 7
2.3 Why PythonCM is primary 7
2.4 Why Slurm still exists 8
3. Racks and exact-node readiness 9
3.1 Rack Validation Status 9
3.2 Rack Detail 10
4. Running validation safely 11
4.1 The standard workflow 11
4.2 The four principal 16K workloads 11
4.3 Single-node versus multi-node evidence 11
4.4 IMEX blockers 12
5. Multi-rack operations 12
6. Scheduler and activity tracking 13
7. Logs, parser results, and proof 14
7.1 Current campaign log locations 14
7.2 What a pass requires 14
7.3 Tail, parser, and download 15
8. Validation Status, attribution, and node-down handling 15
9. Analytics, reports, and handoff artifacts 17
10. Observability 18
10.1 The telemetry chain 18

10.2 Metrics screen and honest failure display 18
10.3 Events and scheduler reasons 19

11. Projects, catalogs, and tools 20
11.1 Projects & Clusters 20
11.2 Platform Catalog 21
11.3 Tool Registry 22

12. API reference and settings 23
12.1 API Reference 23
12.2 Settings 24

13. Health checks and reducing head-node load 24
14. Troubleshooting guide 25
14.1 Rack status is stale or every validation is pending 25
14.2 A log cannot be found 25
14.3 PythonCM command fails 25
14.4 Slurm fallback is pending 25
14.5 IMEX failure 25
14.6 Multi-rack NCCL hangs in initialization 25
14.7 UI source change is not visible 25

15. Safety matrix 26
16. Beginner handoff checklist 26
17. Glossary 27
18. Installation and deployment guide 27
18.1 Deployment topology 27
18.2 Installation inputs 28
18.3 Verify and unpack the release 28
18.4 Secrets and permissions 28
18.5 Configure NscaleTX 29
18.6 BCM/PythonCM integration 29
18.7 Validation scripts and containers 30
18.8 Build and install 30
18.9 Persistent state and mounts 31
18.10 Deployment gate 31

19. First validation acceptance 32

20. Upgrade, rollback, and recovery 32
20.1 Upgrade 32
20.2 Rollback 33
20.3 Common deployment failures 33
21. Security and operational boundaries 33
22. Release evidence checklist 34
23. Screenshot index 35
24. Provenance and current references 35

About this guide

Version 3.0 documents the current release candidate and supersedes the earlier Slurm/cmsh-centered mental model. **PythonCM is the primary provider for inventory, node management, and supported remote commands; Slurm remains an explicit, bounded source for scheduler state/reason and an on-demand fallback for workloads whose validated semantics depend on Slurm, Pyxis, MPI, or LLMB.**

This is an operator guide, not merely an engineering record. The first half gets a new user safely through login, navigation, one-rack validation, logs, and failure handling. Later sections explain the provider architecture, observability, administration, and troubleshooting.

The screenshots were recaptured from the live NscaleTX 16K Rack Status deployment on 14 September 2026 using authenticated, read-only browser navigation. Counts and job rows will change; labels, routes, controls, and workflows are the evidence to follow. No secret, password, token, or private key is printed in this guide.

What changed in this release

Area	Guide v1.1 model	Guide v2.0 model
Recurring rack status	cmsh inventory plus recurring Slurm queries	PythonCM inventory/topology and a local durable activity overlay; no recurring Slurm or cmsh command on the status path
Rack grouping	Rows inferred from hostname/ coordinates	Authoritative BCM PythonCM NodeGroups displayed as SU headings; real hostnames still determine logical rack IDs
Node lifecycle	Mostly Slurm/controller commands	PythonCM for drain, resume, power, supported ProgramRunner actions, UUID status, and UUID cancellation
Validation jobs	Slurm treated as the general scheduler	Provider is explicit per activity. Coordinated NCCL, NeMo, Llama, ClusterKit, NVLoom, and MNUberGEMM recipes retain targeted Slurm/transport fallback where PythonCM is not semantically equivalent
History	Live process/job views could be mistaken for durable proof	Activities, provider, exact nodes, child IDs, logs, and outcomes are retained in a durable registry
UI	Earlier single-rack navigation	Domain navigation: Compute, Infrastructure, Observability, Tools, Analytics & Benchmarks, Platform, Settings
Validation result	Results spread across log views	A first-class Validation Status floor map for 14 SUs and 4,032 campaign nodes, with domain filters, evidence, Job IDs, scheduler reason, prediction, and CSV/PDF/JPEG exports
Rack node context	Availability lacked the scheduler explanation	Rack Detail displays live Slurm state and reason beside each exact node, without making a fleet-wide scheduler query on every browser refresh
Launch safety	Rack-name expansion could obscure exact allocation	Exact-node campaign snapshots, eligibility gates, pinned workload contracts, per-rack MNUberGEMM jobs, and fail-closed image/staging checks

Five rules to remember

1. **Select exact nodes.** A rack label is not a compute hostname, and a GB300 rack uses a non-contiguous tray pattern.
2. **Preserve evidence before repair.** Record the job/activity ID, validation type, first decisive error, node, and log path.
3. **Missing evidence is not a pass.** Empty, stale, skipped, unreachable, and unparseable are visibly different from green.
4. **Drain only on node-specific proof.** A failed multi-node collective does not prove every allocated node is defective.
5. **Do not confuse provider with feature name.** A legacy `/api/slurm-*` compatibility URL does not mean every operation uses Slurm.

1. Your first ten minutes

1.1 Open and authenticate

The service is normally reached through the approved tunnel at the URL provided by the site operator. The 16K deployment serves the application on the observability/app server at port `5056`; your local tunnel may use a different local port.

1. Open the supplied URL.
2. Authenticate with the approved shared-token or PAM flow.
3. Confirm the top bar says **NscaleTX · 16K** and shows a green **PythonCM** provider badge.
4. Confirm the active project on the Overview page is the NscaleTX 16K project before launching anything.

> Never place an access token, SSH key, BMC password, or terminal history in a screenshot, ticket, or guide.

1.2 Read the Overview page

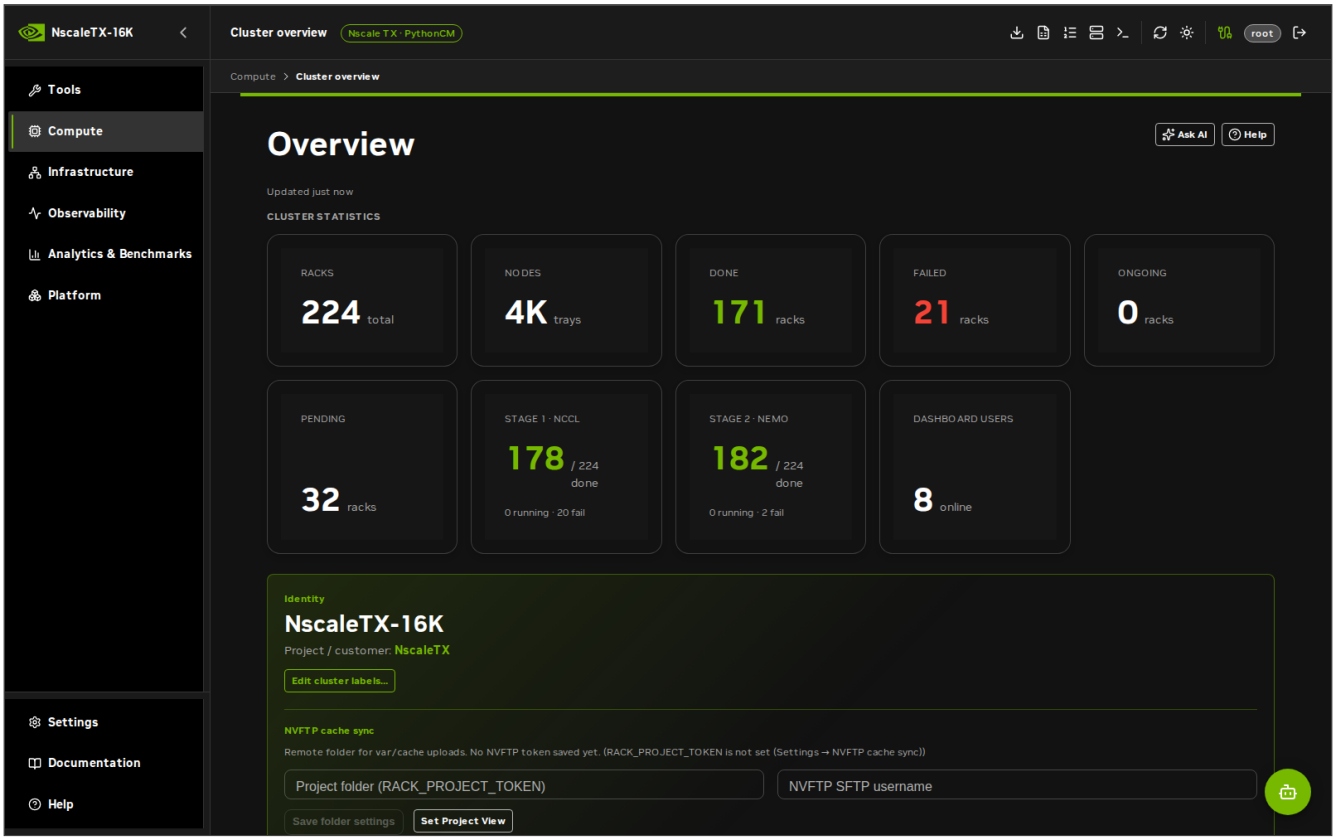


Figure 1. Current 16K Cluster Overview. Counts are live and will change. The provider badge and active project are the important orientation checks.

The cards answer broad questions: how many racks and nodes are in scope, how many are available, how many validation activities are running, and whether the active cluster identity is correct. This page is an overview, not exact-node readiness proof.

1.3 Know where things live

Sidebar domain	Use it for
Compute	Cluster overview, nodes, racks, multi-rack selection, scheduler/activity tracking
Infrastructure	Ethernet, IB/NVLink, and storage visibility
Observability	Metrics, validation logs, and current events
Tools	Approved operations/scripts, workflows, terminal
Analytics & Benchmarks	Parsed results, post-run node attribution, generated reports
Platform	Projects/clusters, hardware catalog, tool registry, API reference
Settings	Runtime configuration and administrative settings

2. The 16K cluster mental model

2.1 SU, rack, tray, node, and GPU

The NscaleTX fleet is organized into **super-units (SUs)**. PythonCM NodeGroup membership supplies the SU heading shown in Rack Status. A logical compute rack is derived from the real compute hostname, not the BCM physical-coordinate rack object.

Examples:

- `SU12` contains logical racks such as `ink50-dh01-a05`.
- A full GB300 NVL72 rack normally contains **18 compute trays**.
- Tray suffixes are non-contiguous: commonly `11–18` and `28–37`.
- Each compute tray exposes four GPUs, so a full rack represents 72 GPUs.

Do not invent `<rack>-01` through `<rack>-18`. Rack Status resolves exact real nodes from PythonCM and the active naming templates.

2.2 Inventory facts versus campaign denominators

The inventory count shown by the live UI is the current PythonCM-resolved scope. A validation program may use a separate approved denominator or roster (for example a 4,032-node acceptance campaign). Do not force the live inventory count and a campaign denominator to match without checking exclusions, non-compute entities, commissioning state, and the campaign manifest.

2.3 Why PythonCM is primary

PythonCM is BCM's supported object/API path for entity inventory and management operations. It provides structured PhysicalNode, Rack, and NodeGroup data and preserves native UUIDs for ProgramRunner commands.

The recurring status flow is intentionally lightweight:

```
Browser (30-second refresh)
        |
        v
Rack Status cached canonical snapshot
        |
        +-- PythonCM inventory / NodeGroup topology / device state
        +-- bounded scheduler node-state/reason overlay
        +-- local durable activity registry
        +-- bounded log parser results

No `journalctl` or systemd-journal scan is part of browser refresh. Scheduler
state/reason is obtained through the provider-neutral endpoint and cache; commands that
launch, cancel, drain, resume, or repair remain explicit operator actions.
```

2.4 Why Slurm still exists

PythonCM ProgramRunner is not a translation layer for a coordinated MPI allocation, Pyxis container scheduling, `llmb-run submit`, or a pinned multi-node validation recipe. Replacing those with independent commands would change what the test proves.

Therefore the application is **PythonCM-primary with explicit feature-specific fallbacks**:

Operation	Primary provider
Inventory, SU topology, node state/reason	PythonCM
Drain/resume and approved power operations	PythonCM
Supported exact-node commands, hardware/NVLink checks	PythonCM ProgramRunner
NCCL, NeMo, Llama, ClusterKit, NVLoom, MNUberGEMM workload recipes	Explicit on-demand Slurm or transport fallback when the accepted recipe requires it
Job/history overlays	Durable Rack Status registry; targeted provider query only when detail is requested
Metrics and logs	VictoriaMetrics, Loki, and bounded local log access

3. Racks and exact-node readiness

3.1 Rack Validation Status

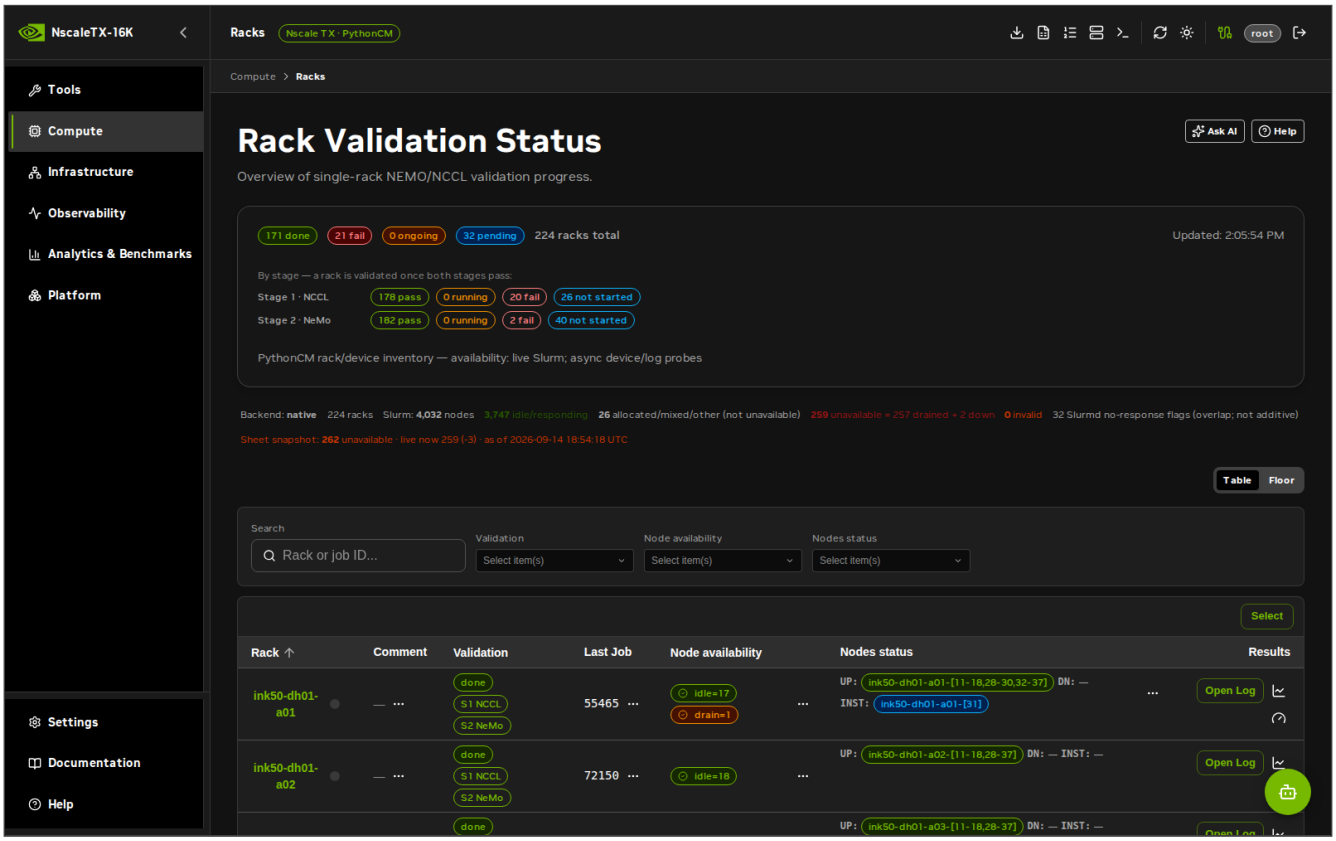


Figure 2. Racks view with stage badges, node-availability summaries, filters, and results actions.

Use this screen to find a rack and understand its current validation stage. Badge meanings:

Badge	Meaning
done	The required parser/result contract was satisfied for the represented stage.
fail	A stage produced terminal failure evidence.
ongoing	An activity is currently running or awaiting reconciliation.
pending	No accepted terminal evidence is available yet.

The page is safe to browse and filter. Refreshing it does not launch a job.

3.2 Rack Detail

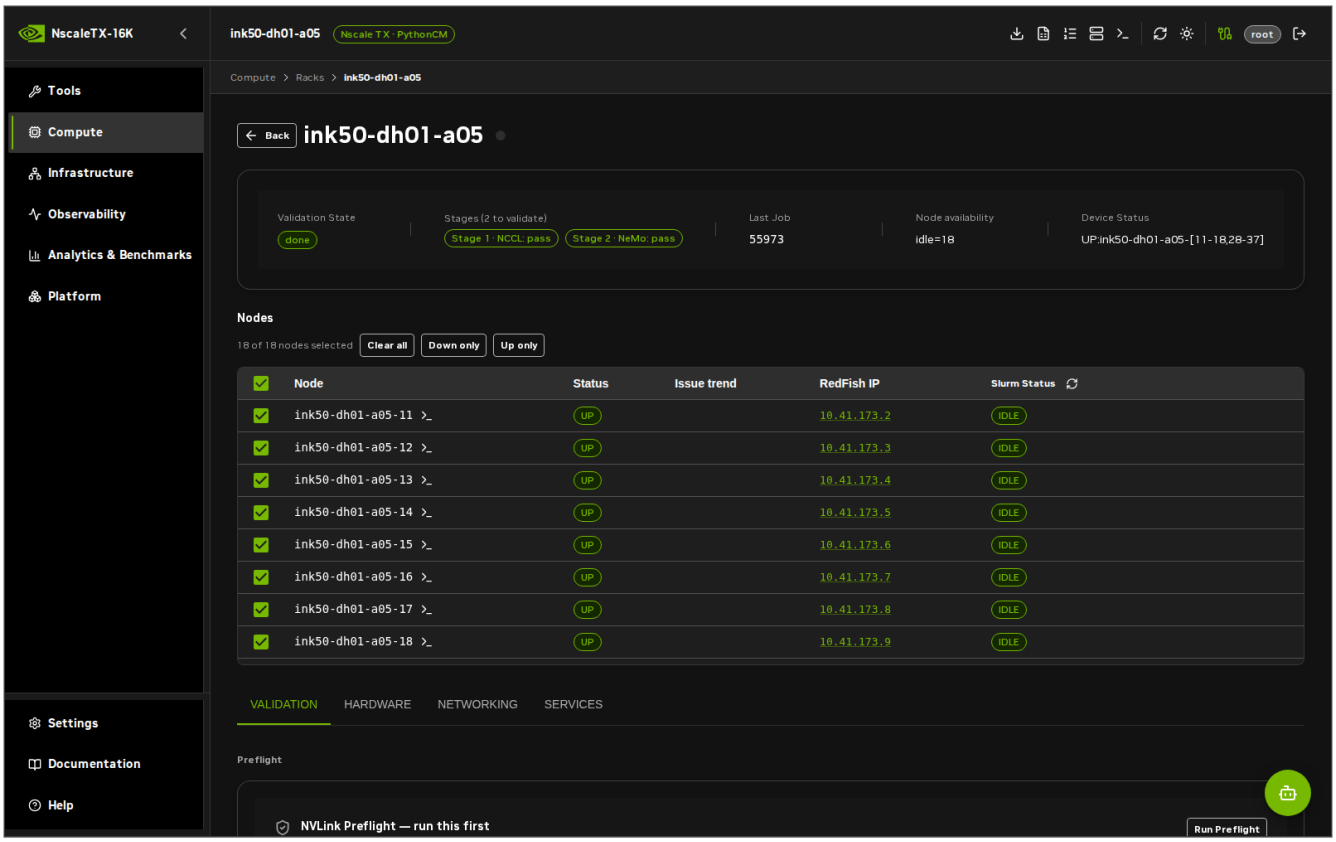


Figure 3. Rack Detail for `ink50-dh01-a05`. The node table is the exact target authority. Its Slurm Status column shows state and places the full reason directly beneath the state when one is present.

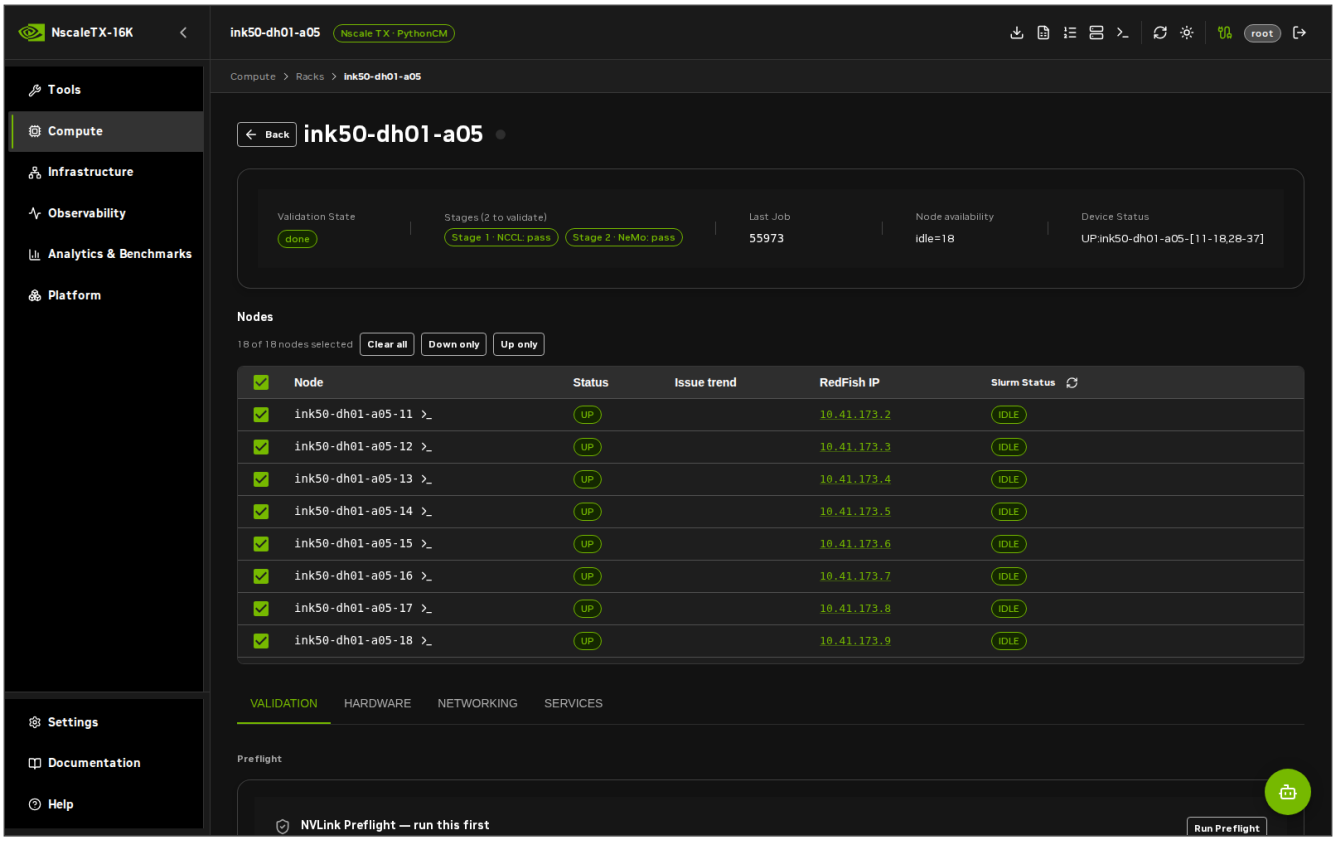


Figure 4. Dedicated scheduler-state/reason capture. An empty reason means the scheduler reported no comment; it must not be replaced with an invented explanation.

Before any test:

1. Confirm the rack ID and SU.
2. Review every node's state and reason.
3. Exclude down, drained, failed, maintenance, or unresolved nodes.
4. Confirm whether the recipe requires a full 18-node rack or allows partial scope.
5. Review the active rack lock and other operators.
6. Run the preparation/dry-run step before launch.

The rack page groups actions into Validation, Hardware, Networking, and Services. Reset, power, drain, undrain, MAC, IMEX, and NVSwitch fix actions can change production state and require evidence plus confirmation.

4. Running validation safely

4.1 The standard workflow

1. Start on **Compute** → **Racks** and confirm the snapshot is fresh.
2. Open one rack and review exact node eligibility.
3. Run a read-only preflight where the recipe provides one.
4. Select the approved preset and storage mode.
5. Review image name/digest, exact nodes, node count, runtime ceiling, and fallback provider.
6. Stage images only through the approved bounded copy service; verify size/hash receipts.
7. Launch once. Record the outer activity UUID and every native child ID.
8. Track the activity in Rack Detail and **Compute** → **Scheduler**.
9. Open logs/results. Confirm the expected node/rank/test count, not only exit code.
10. Preserve the first decisive error before cancelling, draining, or restarting anything.

4.2 The four principal 16K workloads

Workload	What it proves	Important constraint
NCCL	Coordinated GPU collective and fabric performance	Accepted MPI/topology recipe; full-rack proof requires all expected nodes/ranks and collective markers
NeMo 56B	Distributed Nemotron-H 56B FP8 training behavior	Expected training steps, timing/throughput, and terminal result marker must appear
MNUberGEMM	Multi-node NVLink-domain compute/fabric stress	One independent job per physical rack; never combine separate NVLink domains into one job
Llama 3.1 405B FP8	Pinned large-scale LLMB training acceptance	Exact approved scale/recipe and image; multi-rack only

The four logical workloads currently map to three physical container images because NCCL and NeMo share the validated NeMo/NCCL image. Do not rename an image to imply a different recipe.

4.3 Single-node versus multi-node evidence

Single-node passes isolate host-local readiness. They do not prove a coordinated rack fabric. Multi-node tests prove their exact allocation and recipe only. The tracker must keep these as separate rows and separate evidence classes.

4.4 IMEX blockers

Multi-node NVLink work requires healthy IMEX. If an eligible node blocks a validation because of IMEX:

1. Capture the actual error and native job/activity ID.
2. Identify the test type: NCCL, NeMo, MNUberGEMM, or Llama.
3. Preserve the log path.
4. Drain only the implicated node when the evidence is node-specific.
5. Use an exact scheduler/node reason such as:

```
Drained for IMEX readiness failure - for Job <id> during <test> due to <first decisive error>
```

1. Update the Node Down sheet and Rack Status evidence after the controller accepts the reason.

5. Multi-rack operations

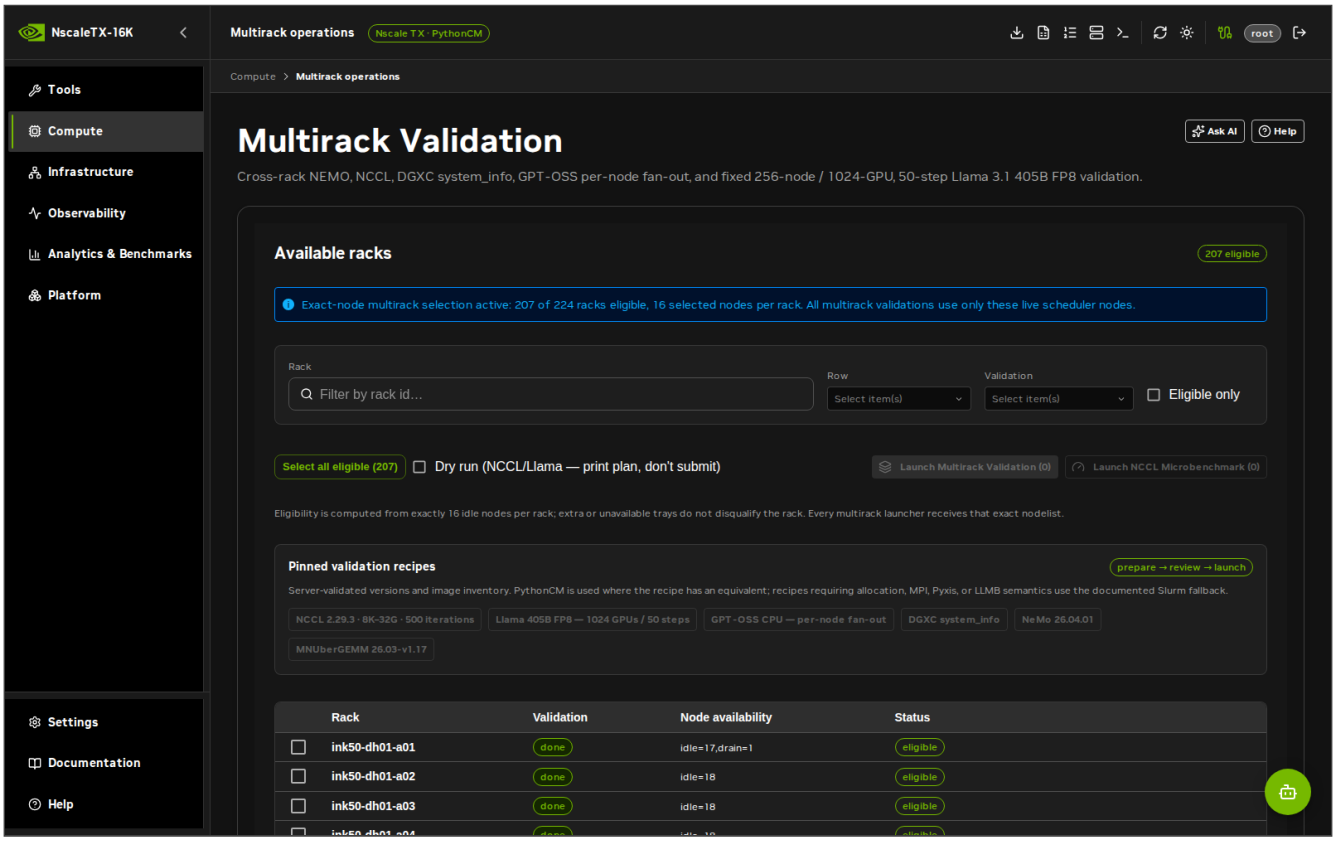


Figure 4. Multi-rack selection and approved campaign controls.

Use **Compute → Multirack operations** for any campaign spanning racks.

1. Filter by SU/rack and eligibility.
2. Select the intended racks.
3. Read warnings about selected racks hidden by a filter.
4. Confirm the exact resolved node total.
5. Run dry-run/readiness.
6. Confirm the provider and fallback reason.
7. Launch and retain the outer activity UUID plus every child job ID.

Special rules:

- MNUberGEMM is an NVLink-domain test: one job per rack.
- A passing single-rack NCCL result does not prove multi-rack communication.
- Llama 405B must satisfy its exact approved campaign scale and model recipe.
- Never shrink a hostlist silently to make a blocked test launch.

6. Scheduler and activity tracking

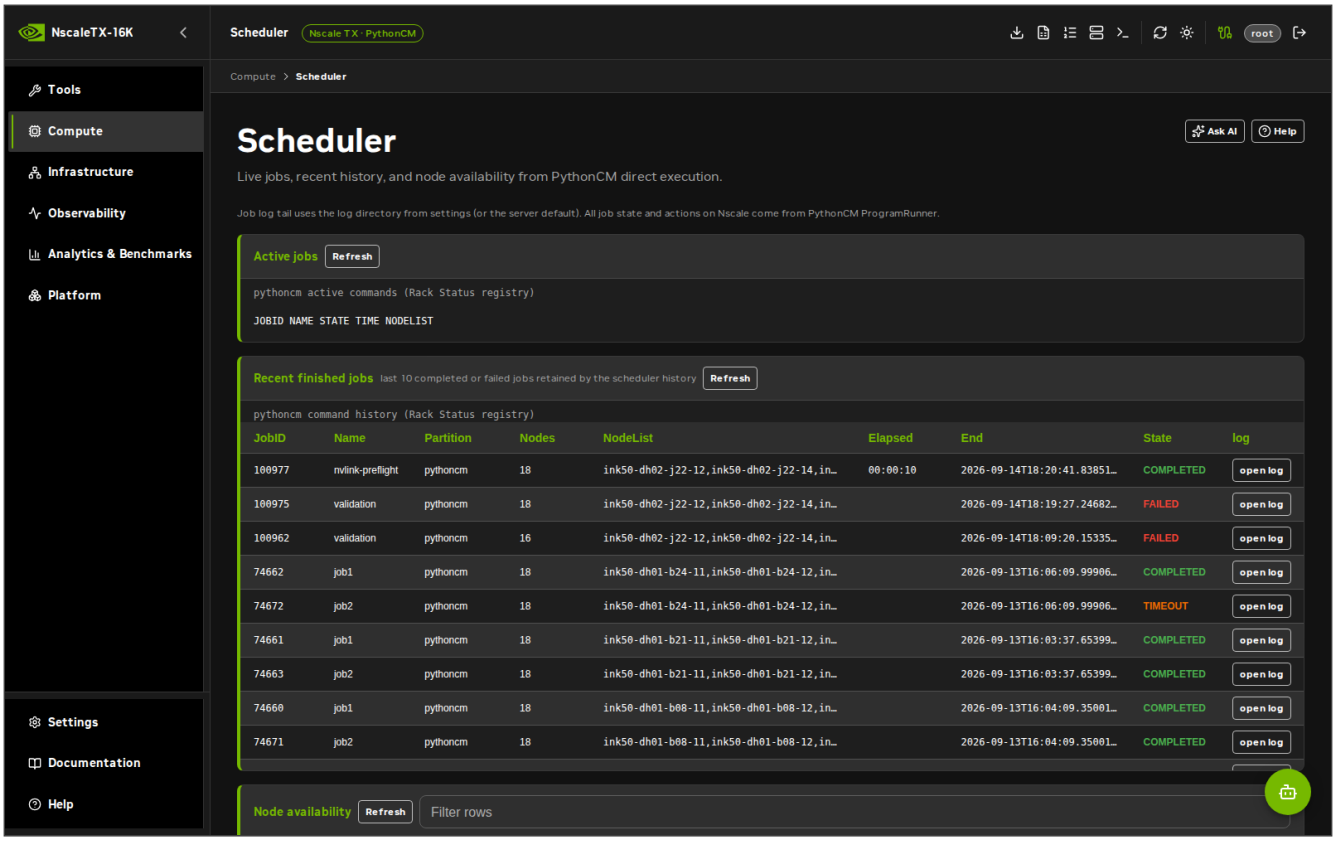


Figure 5. Provider-neutral Scheduler view. UUIDs and numeric IDs retain their native provider provenance.

The Scheduler view joins active/recent activities from the durable registry. Interpret IDs correctly:

- A PythonCM ProgramRunner command has a native UUID.
- A Slurm fallback job has a numeric job ID.
- A Rack Status outer activity has its own UUID and can own multiple native children.

Cancelling an outer activity should cancel its wrapper and every non-terminal child. A cancellation does not mean the hardware failed. Record why it was cancelled.

Use targeted detail/output actions. Rack Status does not need a fleet-wide Slurm poll to display registered fallback work.

7. Logs, parser results, and proof

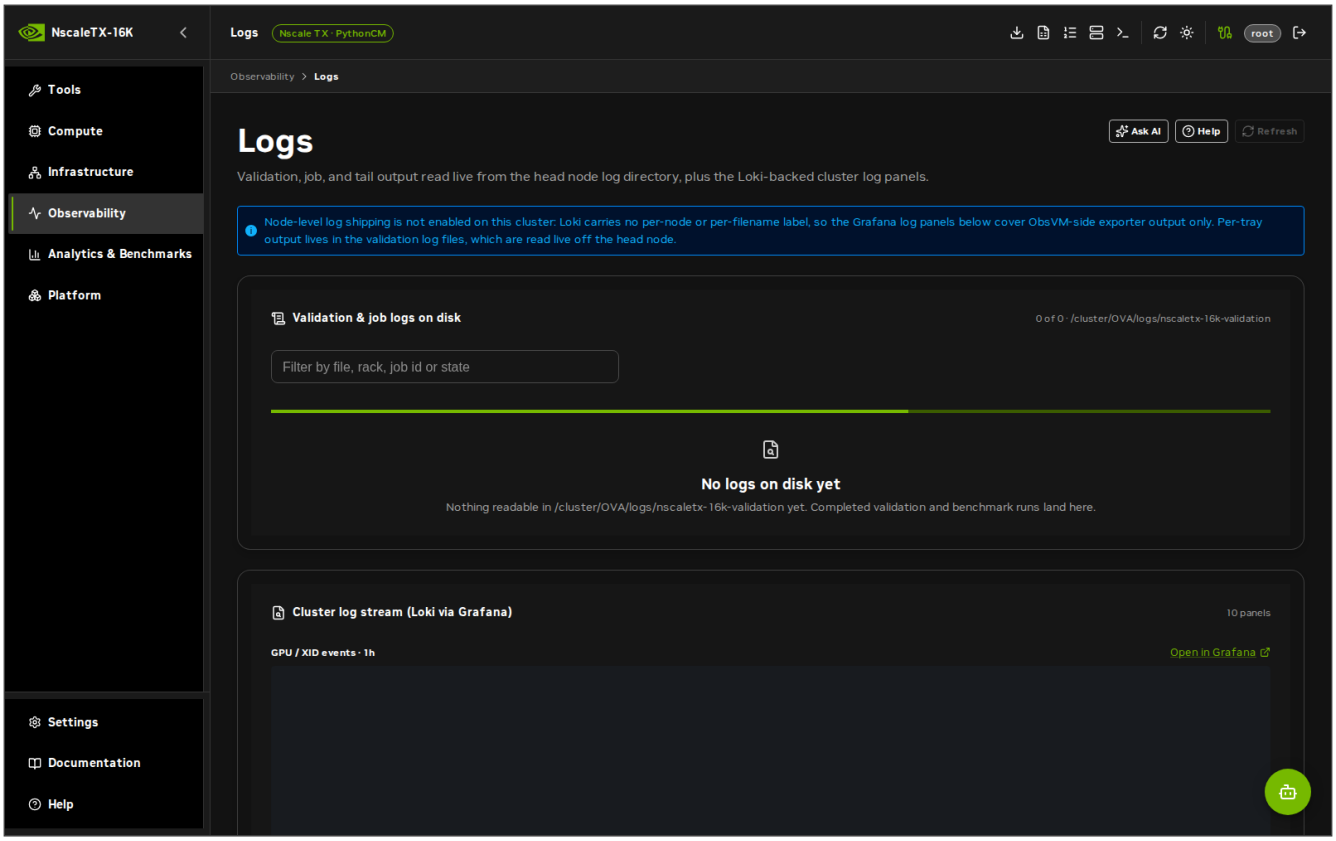


Figure 6. Logs view with validation type, rack, job, state, preview, tail, and open actions.

7.1 Current campaign log locations

Legacy canonical logs may exist under the configured Rack Status validation root. Current SU9–SU14 campaign logs also live in deterministic paths such as:

```
/cluster/OVA/logs/SU12/rack05/multiNodeNCCL/
/cluster/OVA/logs/SU12/rack05/multiNodeNeMo/
```

Examples:

```
nvftp-nccl-five-collective-28709.out
nemo-26.04.01-19774.out
```

Rack Status resolves both the legacy canonical tree and the deterministic SU/rack campaign paths. A full-rack filter is evidence-based: the log must declare/confirm 18 nodes; a filename alone is not sufficient.

7.2 What a pass requires

A green scheduler state alone is insufficient. Confirm:

- exact rack and node allocation;
- expected ranks/GPUs;
- every required collective or training step;
- terminal recipe-specific marker;
- no decisive error hidden before the final line;

- parser result tied to the same job/activity ID.

7.3 Tail, parser, and download

- **Tail** follows current output and is useful while a job is active.
- **Parser results** transforms supported NCCL/NeMo output into structured tables/charts.
- **Open/Download** preserves the source artifact.

If the GUI reports “no matching log,” verify the recorded stdout path, deterministic SU/rack path, exact job ID, permissions, and full-rack node markers. Do not move or rename evidence just to satisfy a stale basename convention.

8. Validation Status, attribution, and node-down handling

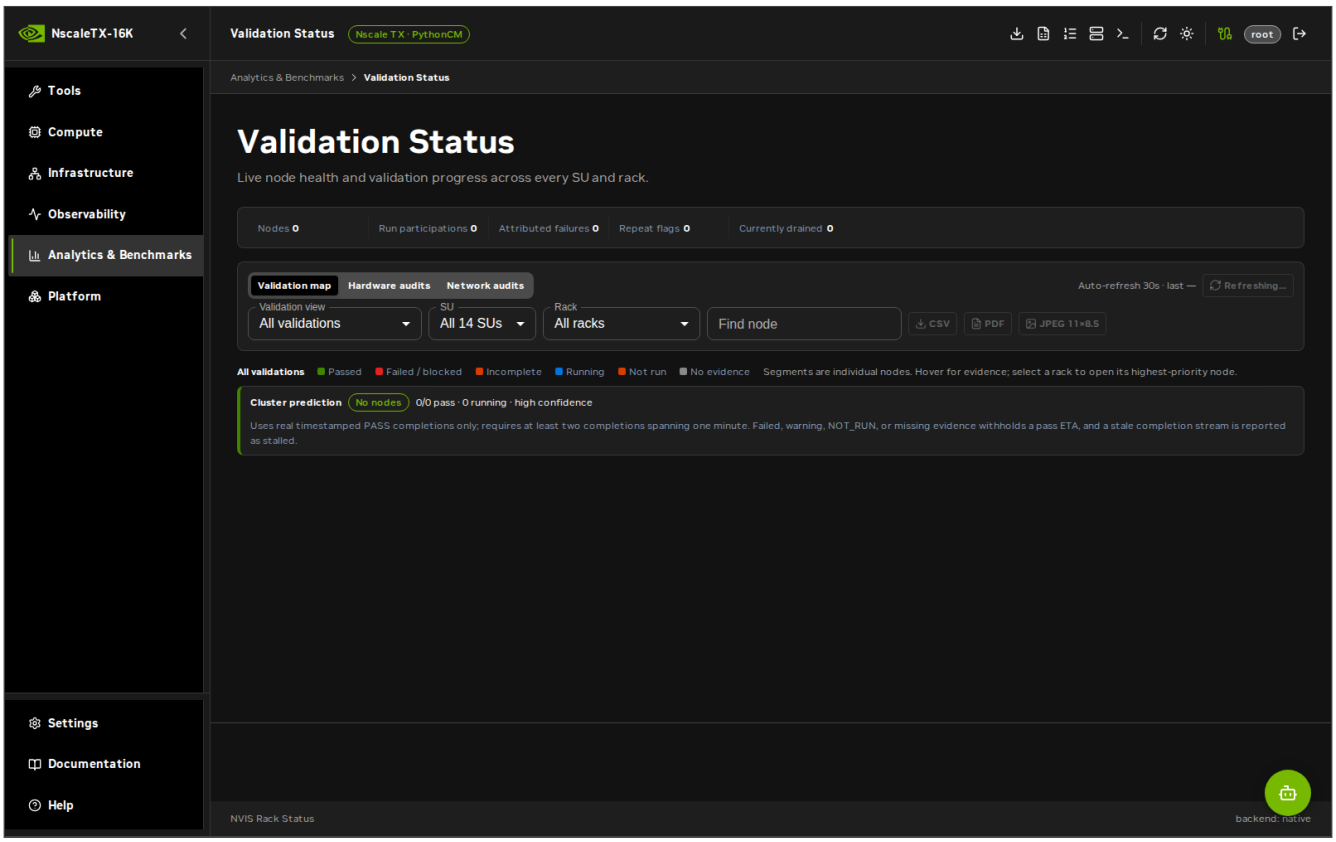


Figure 8. Validation Status floor map. The live campaign view spans 14 SUs and 4,032 exact nodes. Each rack tile contains its node segments; red here means failed/blocked evidence, not an inferred physical defect.

Validation Status joins bring-up results, durable validation activities, exact scheduler targets, DCGM/health evidence, current node state/reason, deep-audit findings, and operator findings. Its domain selector separates **All validations**, Node Health, DCGM, NCCL, NeMo, Llama, MNUberGEMM, and discovered source-specific tests.

Use the filters in this order:

1. Choose the validation domain whose contract you are investigating.
2. Narrow to an SU or rack when required; **All** is appropriate for fleet review.
3. Search for an exact node when responding to an incident.
4. Hover a segment for the evidence summary, then select its rack tile to open the highest-priority node.
5. Read the test/check, raw status, metric, Job ID, evidence/failure reason, timestamp, provenance, and current scheduler comment together.
6. Use CSV for machine-readable handoff, PDF for review, and JPEG 11×8.5 for a fixed floor-map snapshot.

The cluster prediction is deliberately conservative. Only fresh, timestamped PASS completions count toward a pass estimate; failure, warning, NOT_RUN, missing, or stalled evidence blocks a pass ETA. **No evidence is unknown, not healthy.**

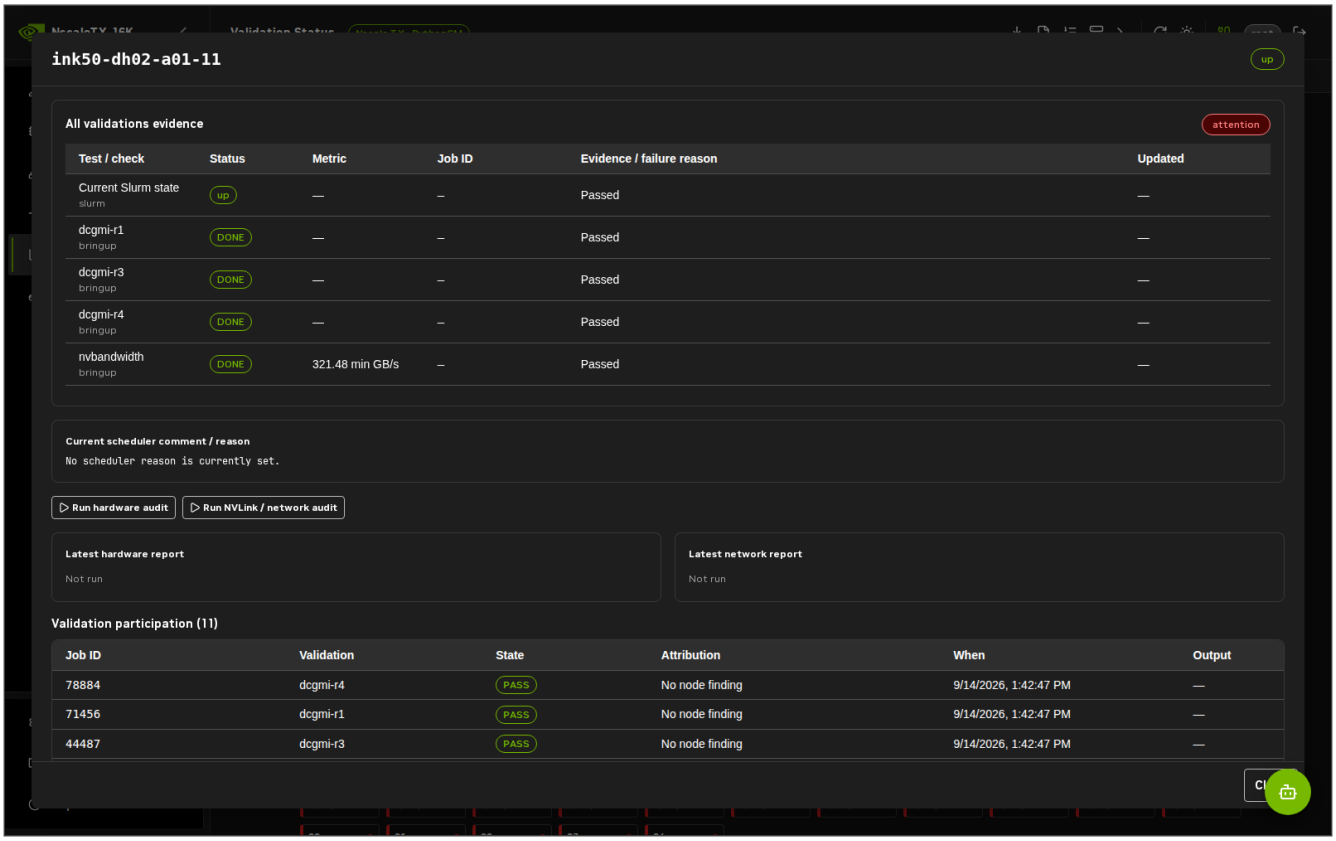


Figure 9. Selecting a floor tile opens exact-node evidence, including check status, source, Job ID, failure reason/provenance, and the current scheduler comment. Audit and drain controls shown here mutate state; do not click them during read-only investigation.

A multi-node collective failure is run-level until evidence identifies a node. Attribute a failure only when:

- structured node-local output names the node;
- the job was single-node;
- the first fatal Xid/ECC/PCIe/DPC/storage/fabric signature identifies it;
- a controlled counter delta or exact-node reproduction proves it.

For every drained node, record:

Required field	Example
Node	`ink50-dh01-a05-33`
Test type	`NCCL full-rack`
Job/activity	`28709` and outer activity UUID
First decisive error	Exact error text, not “test failed”
Evidence path	Source log and any node-local report
Action	Drain/cancel/reset and operator/time
Rerun	Pass/fail/inconclusive and new job ID

Update both Rack Status and the Node Down Google Sheet. The Sheet is a published operational view; durable Rack Status evidence remains the detailed source.

9. Analytics, reports, and handoff artifacts

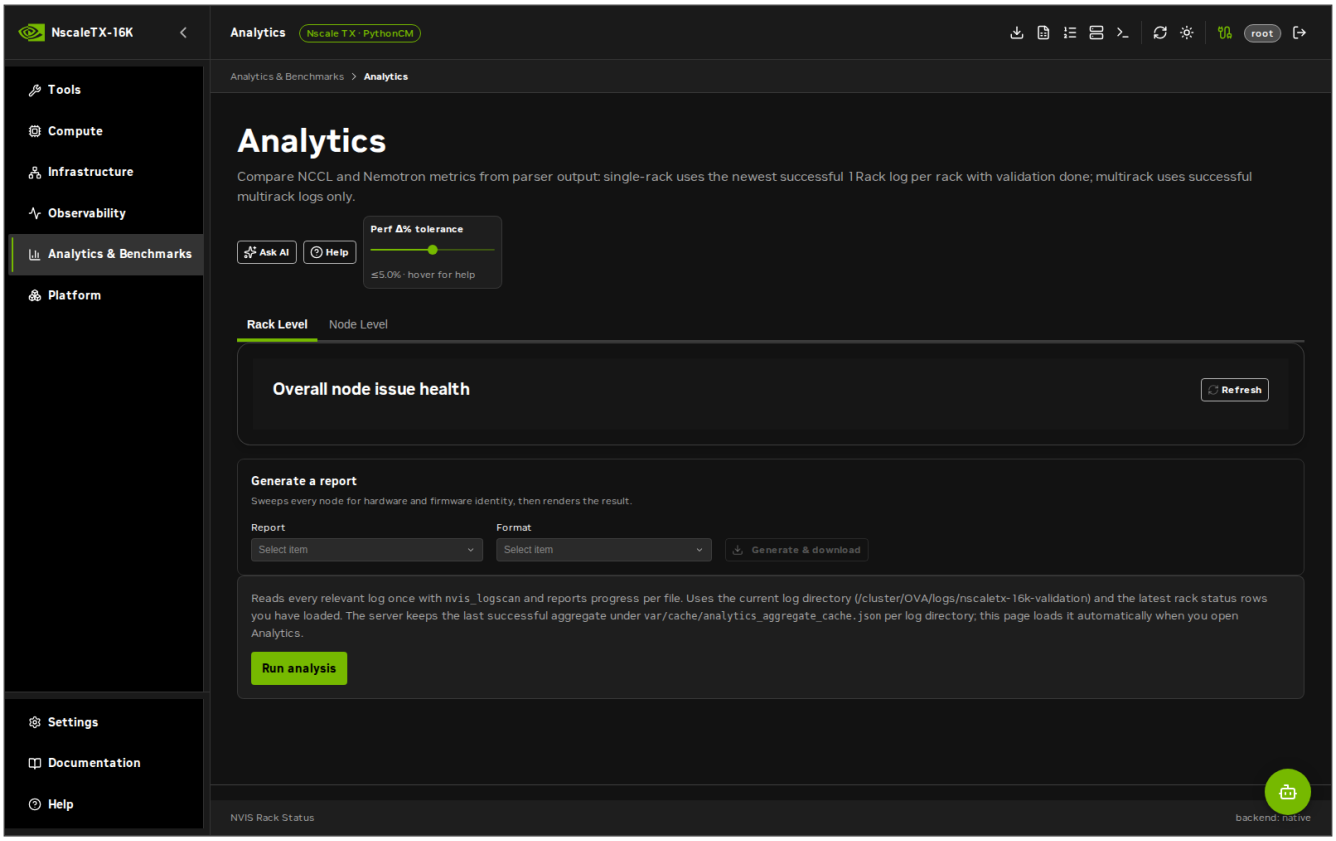


Figure 10. Analytics Overview provides parsed job/result analysis. Aggregate only the intended bounded source and keep the originating log with every chart.

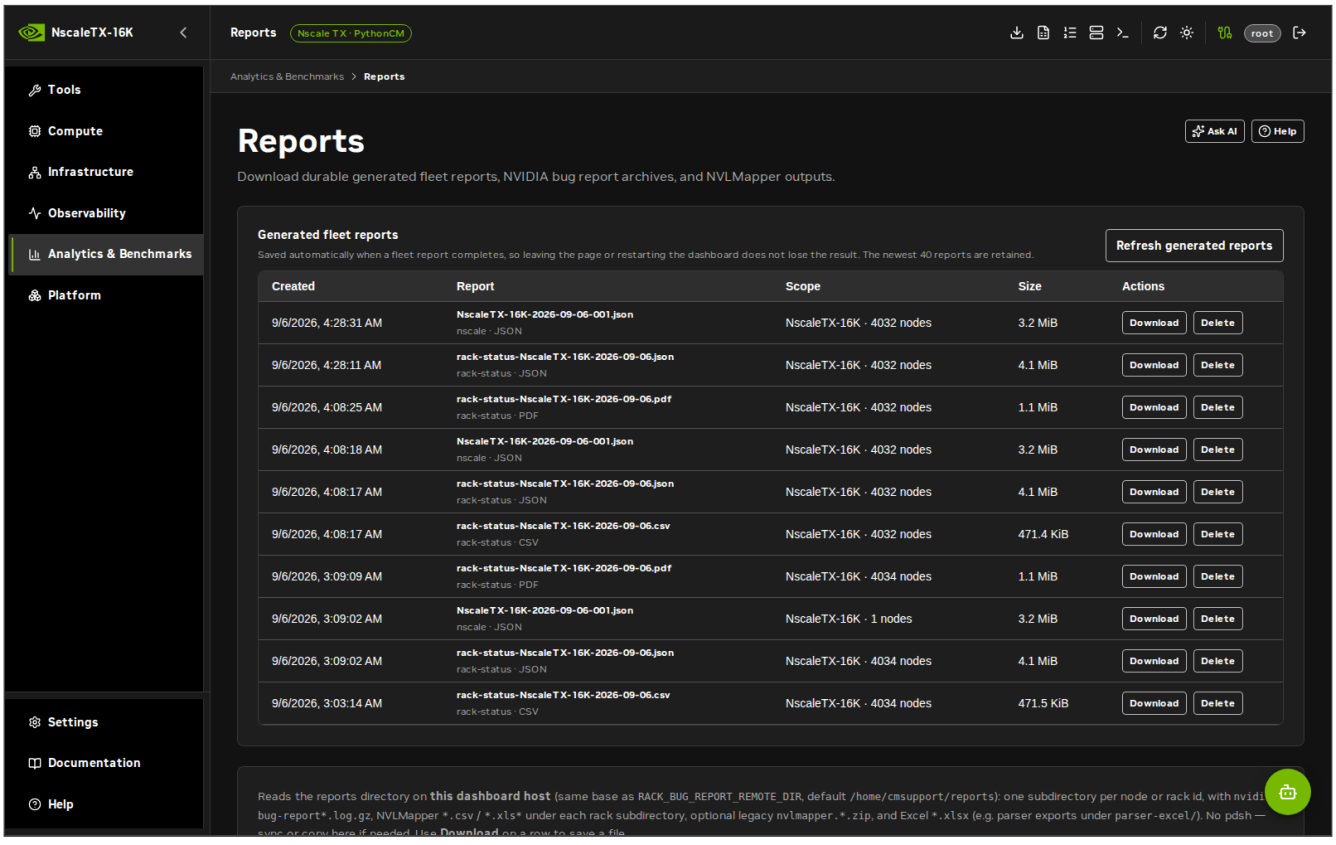


Figure 8. Generated Reports and retained support artifacts.

Generated reports live in the durable application volume. A container restart must not erase them. Each handoff should include UTC start/end, exact target, provider, IDs, image/digest, expected/received counts, first decisive error, actions, rerun result, and evidence paths.

Do not delete a report merely because a later run passed. Preserve the failure-to-recovery chain.

10. Observability

10.1 The telemetry chain

Every compute node should expose and forward the required telemetry through the approved agent stack:

```
node_exporter + cgroup exporter + DCGM exporter + Alloy
                      |
                      +--> VictoriaMetrics (metrics)
                      +--> Loki (logs)
```

End-to-end health requires more than active services:

1. Service is active.
2. Expected local listener exists.
3. Alloy identifies the actual node/rack/SU, not the golden-image source.
4. The write-ahead log does not replay stale identity.
5. A fresh exact-node series arrives at the observability VM.
6. A recent Loki stream arrives where configured.

10.2 Metrics screen and honest failure display

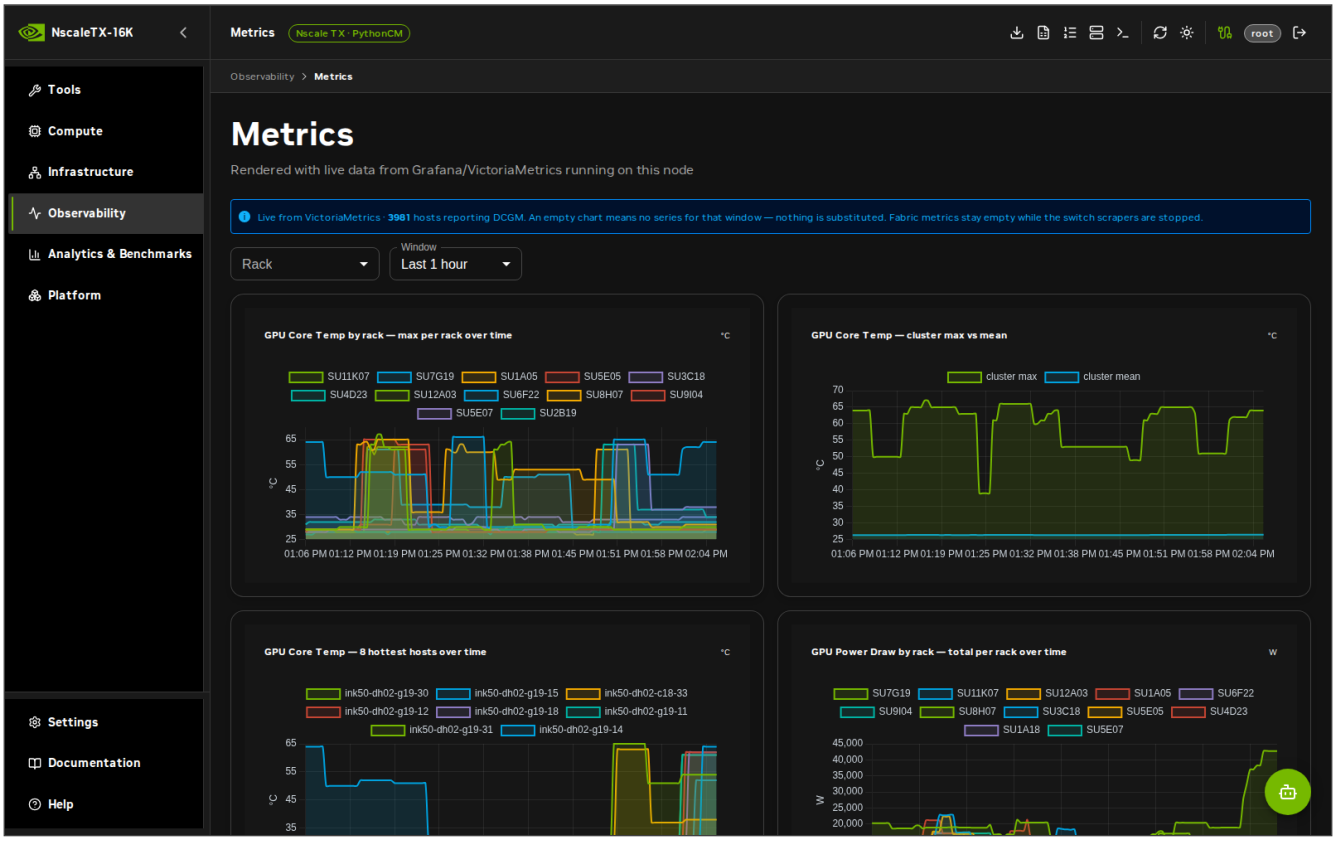


Figure 9. The current Metrics screen honestly displays an upstream query failure. Rack Status does not fabricate a healthy zero when the metrics source is unavailable.

If a chart is empty or failed, check the backend/upstream error, exporter listener, Alloy forwarding, label identity, and exact series freshness. Do not substitute another rack's data.

10.3 Events and scheduler reasons

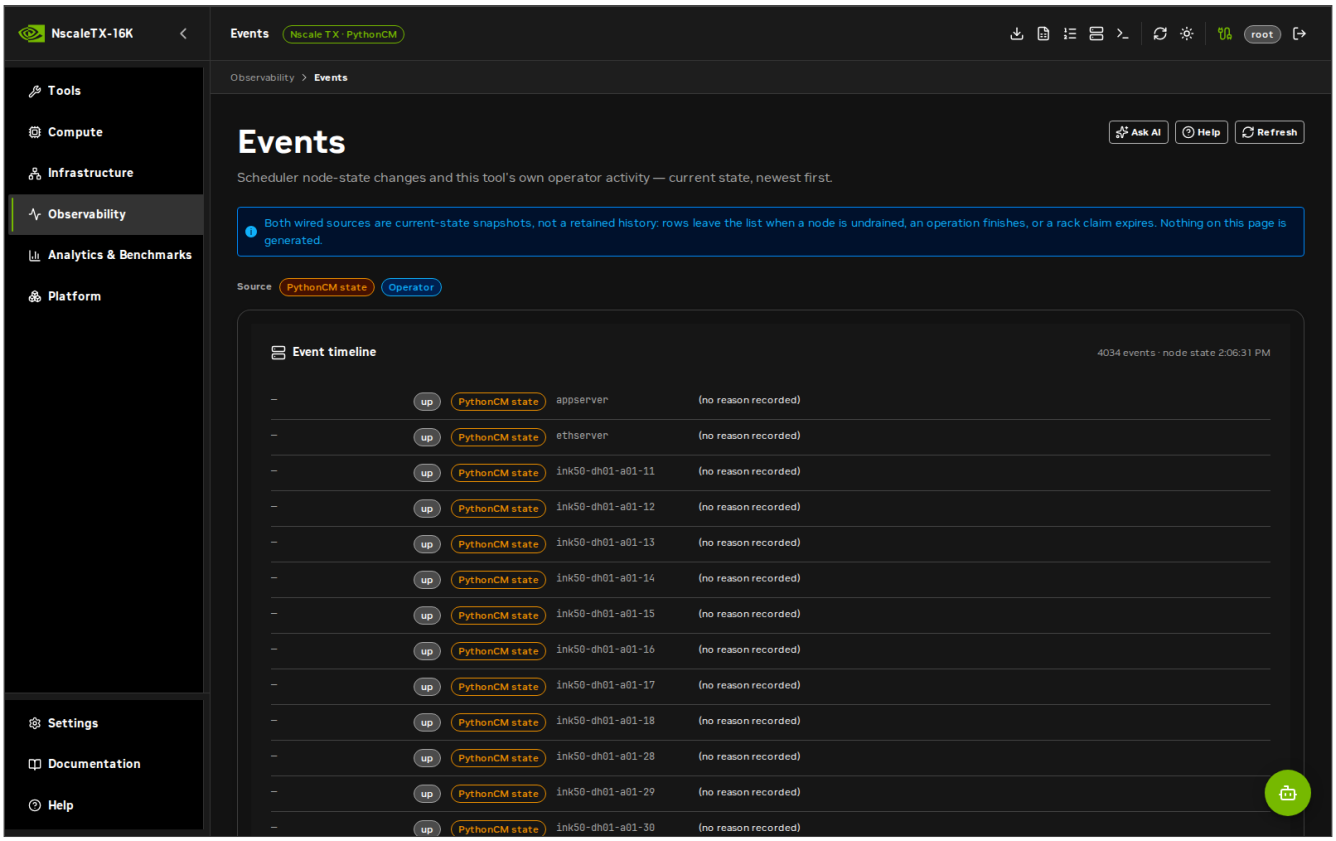


Figure 13. Events joins node reasons and Rack Status activity events. The source and timestamp determine whether an item is current evidence.

Rack Status does **not** scan `systemd-journald` as the browser user. Platform service monitoring remains the responsibility of the site's approved NHC/Alloy monitoring path. A failed local `sudo` invocation can generate sudo security mail even when no head-node command ran; the mail relay destination must not be mistaken for the event origin.

11. Projects, catalogs, and tools

11.1 Projects & Clusters

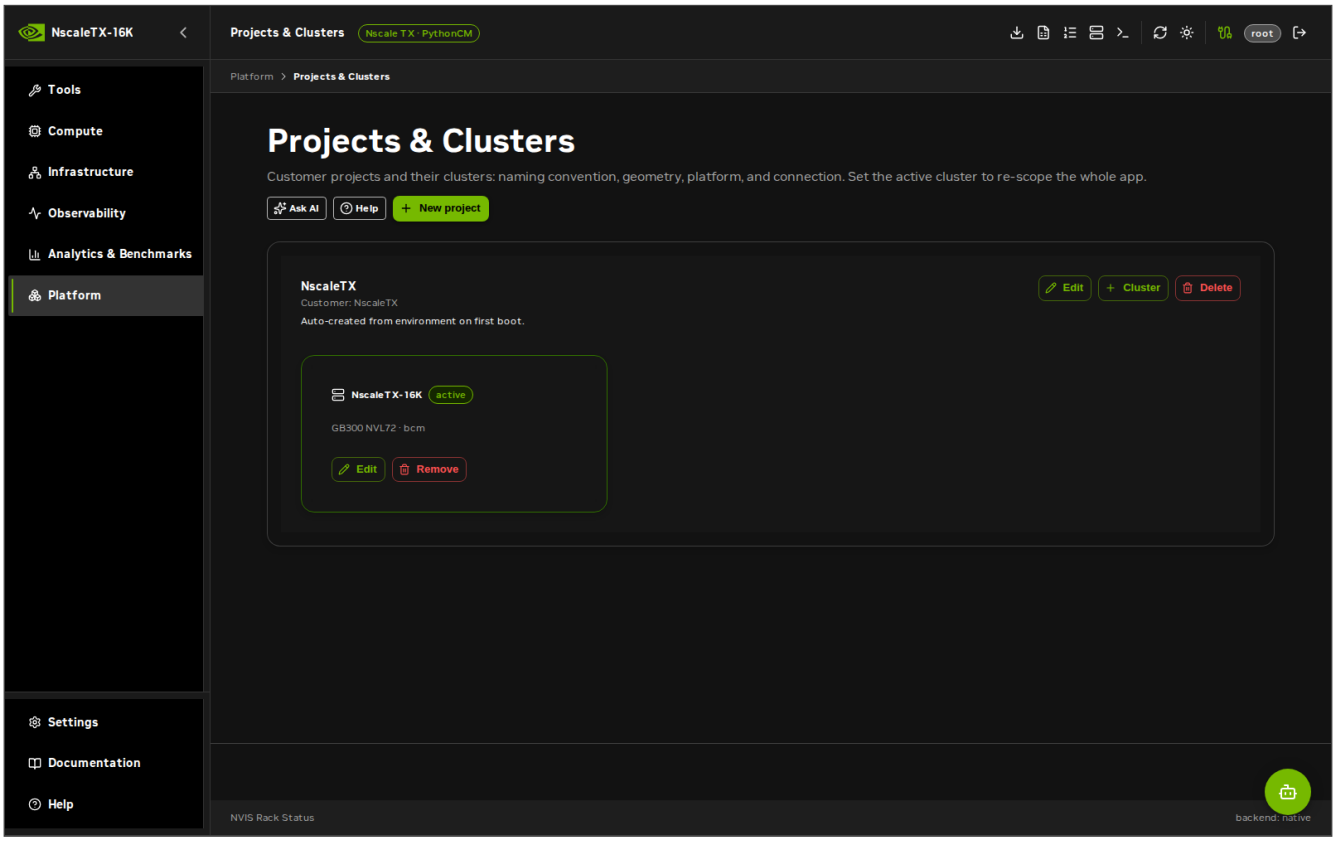


Figure 10. Active project/cluster configuration. Changing active cluster changes the scope of the entire application.

The active NscaleTX project carries cluster identity, platform, naming, connection, and provider configuration. Confirm this page before administrative work. Do not allow a stale generic catalog to override the isolated NscaleTX catalog.

11.2 Platform Catalog

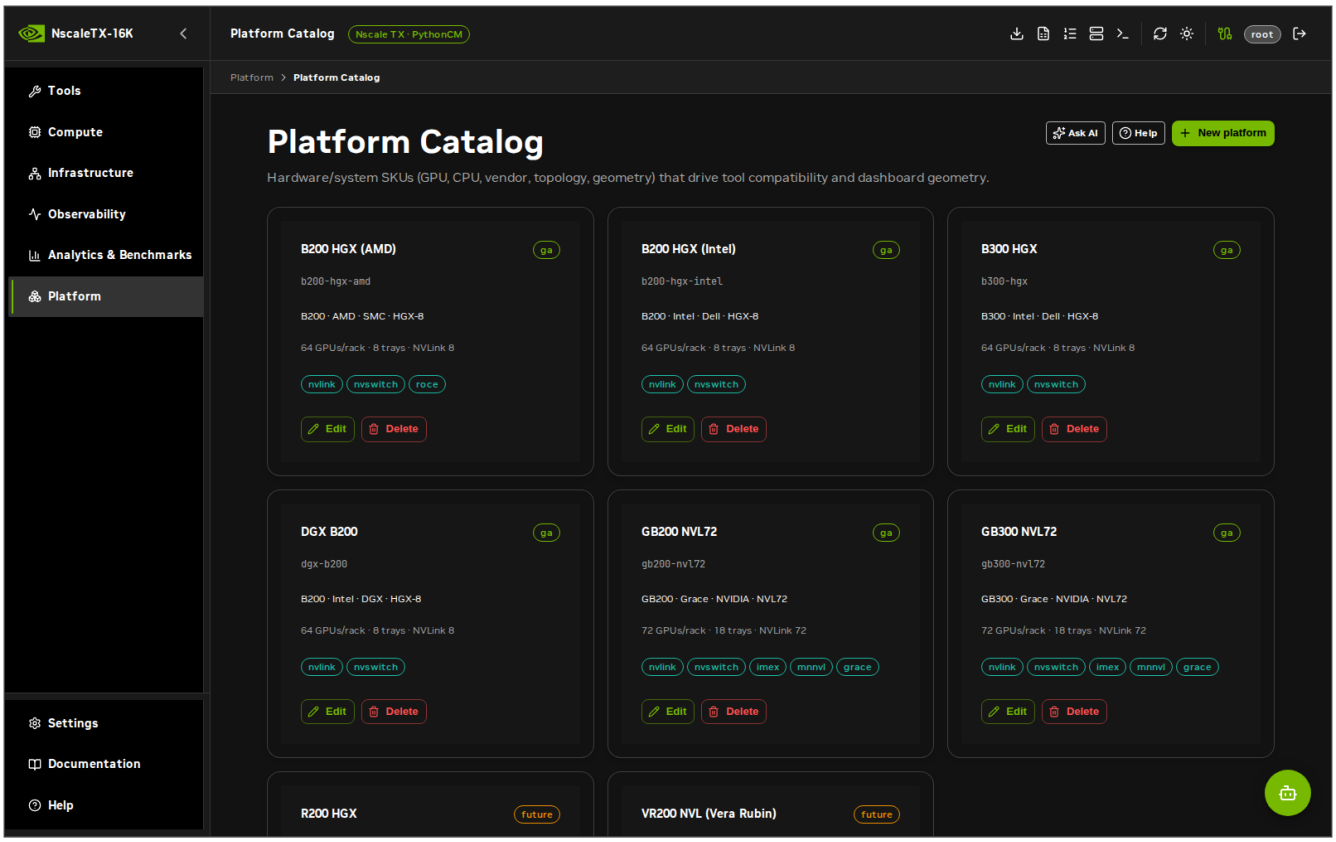


Figure 11. Hardware platform definitions and capability metadata.

The platform catalog describes GPU/server generation, rack geometry, capabilities, and tool defaults. It is not live inventory. Editing it changes planning/display behavior and must be reviewed like code.

11.3 Tool Registry

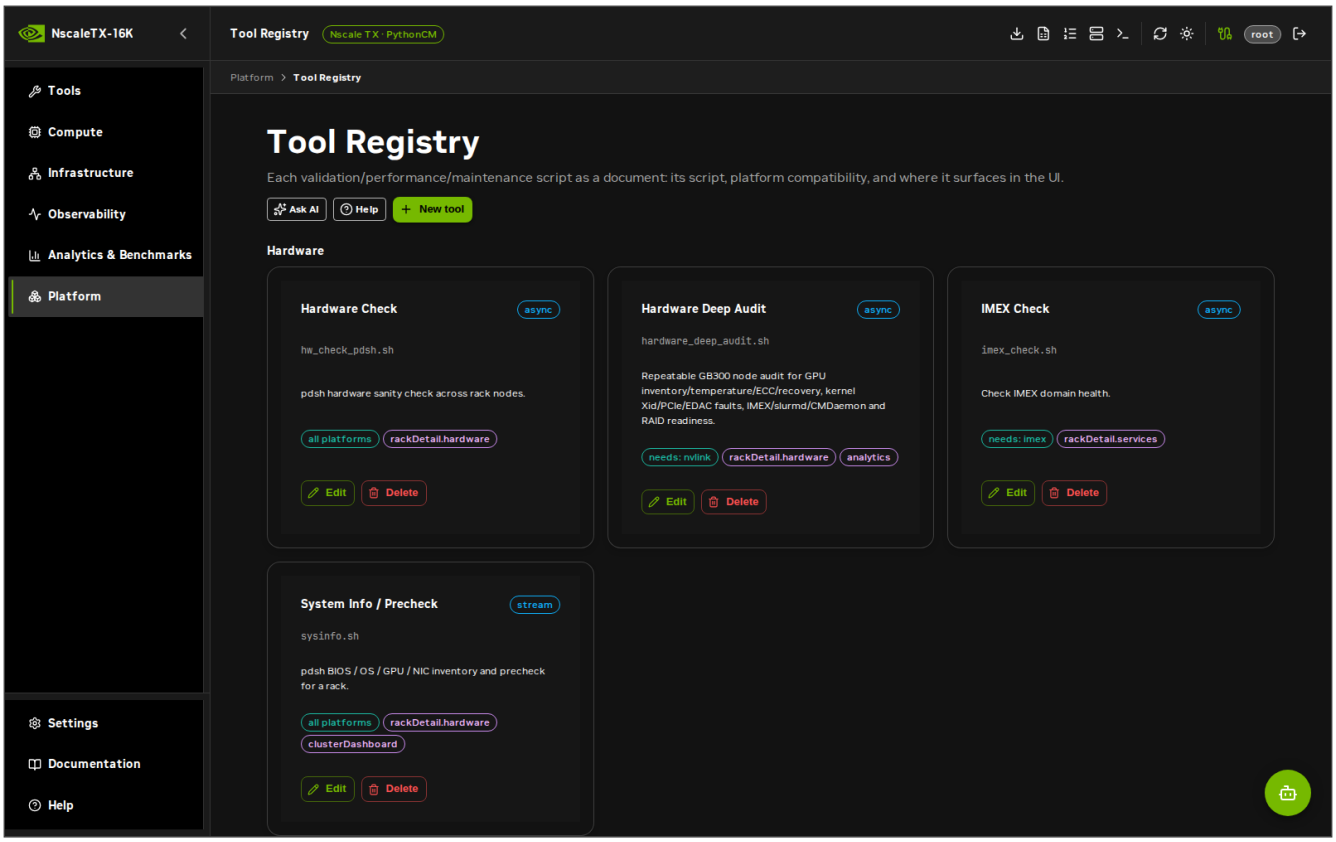


Figure 12. Tools are first-class catalog entries with compatibility, execution mode, surfaces, and defaults.

New tests and scripts belong in the tool/catalog model rather than an untracked hard-coded list. A tool definition does not prove its external binary/image exists on every node; readiness still checks the target.

12. API reference and settings

12.1 API Reference

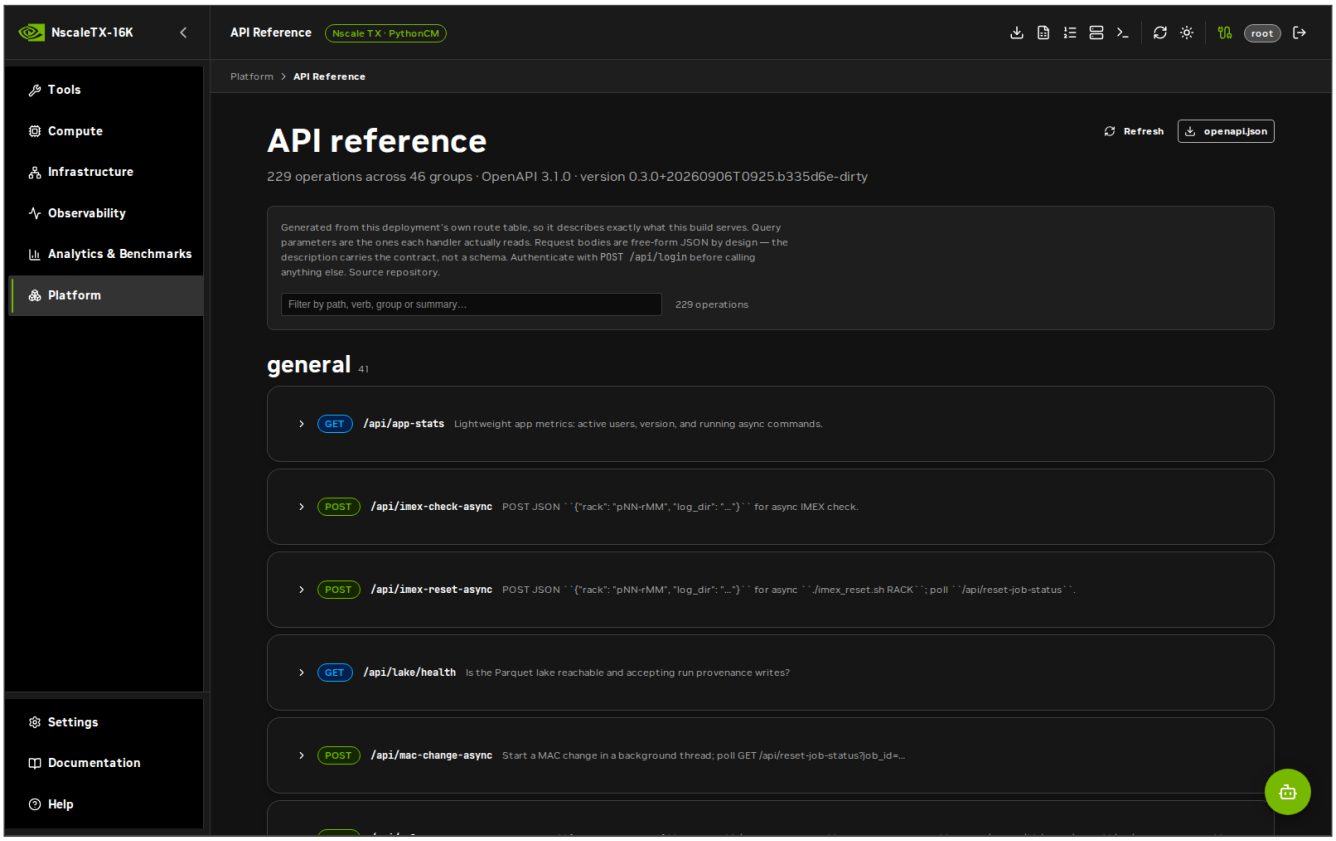


Figure 13. Live registered API reference. Use this when static documentation and the deployed route table disagree.

The live route table is authoritative for the running build. Provider-neutral scheduler routes are preferred. Historical Slurm-named aliases remain compatibility contracts and do not select the runtime provider.

12.2 Settings

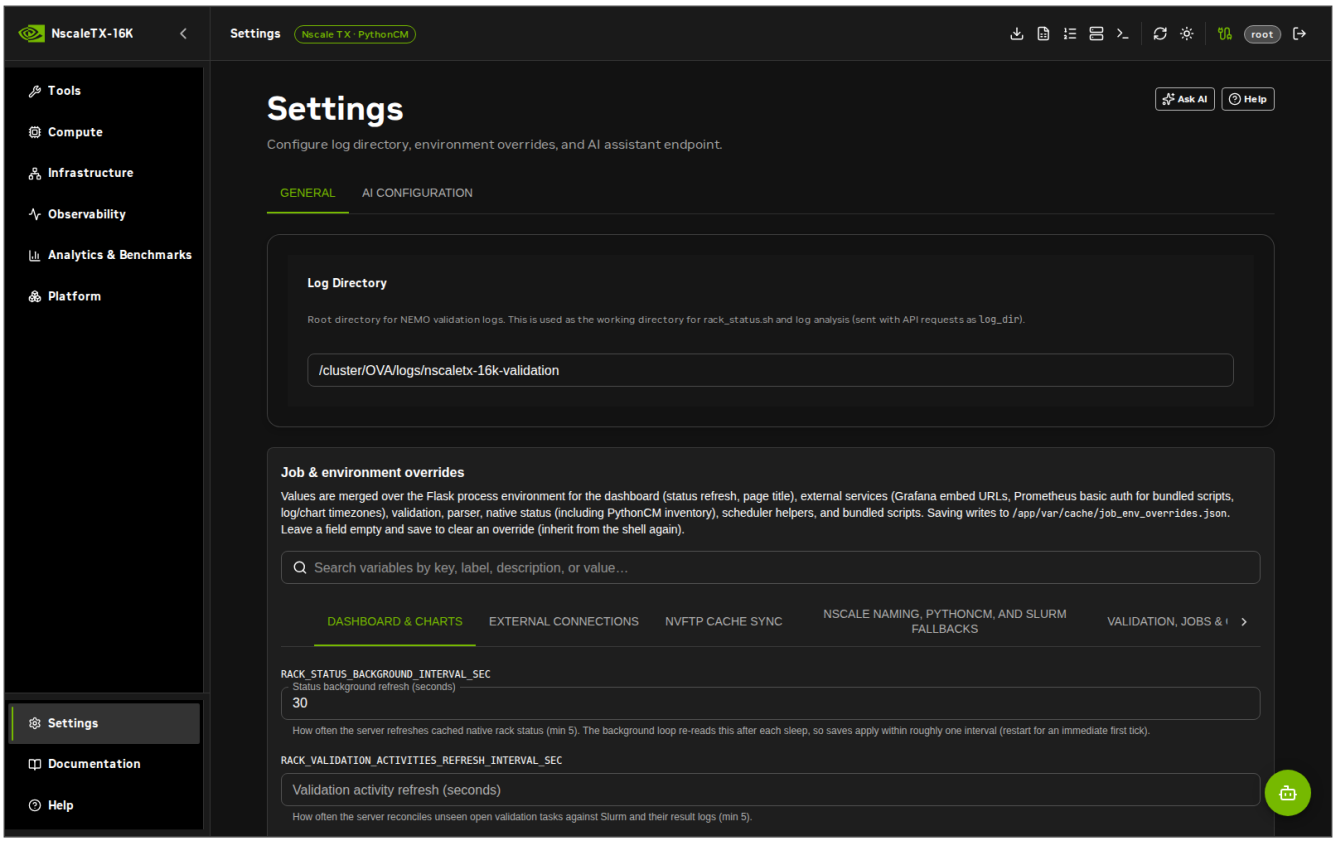


Figure 14. Runtime settings and override search.

Settings overrides can supersede deployment defaults. Record who changed a production override, when, and why. Secrets stay server-side and must not appear in frontend fields, artifacts, or screenshots.

Important production identity:

```
RACK_CLUSTER_ID=nscale-tx
RACK_CLUSTER_NAME=Nscale-TX
RACK_CLUSTER_CUSTOMER_NAME=Nscale
RACK_CLUSTER_PRODUCT_LINE=gb300
RACK_PLATFORM=bcm
RACK_SCHEDULER=pythoncm
```

13. Health checks and reducing head-node load

Single-node bring-up checks must keep their intended health logic while avoiding unnecessary head-node syslog, logging, and routing load.

Current operating principle:

- the app server can reach compute nodes directly for approved direct-node paths;
- per-node health-check output should be written to the dedicated OVA health-check log tree;
- repeated thousands-of-node permission errors must not be forwarded into head-node syslog;
- pausing `run_jobs` or `run_health_checks` is a temporary load-control action, not the permanent fix;
- changes made in `/cluster/bringup` must be ported to the authoritative repository because in-place edits can be overwritten.

Recommended log root:

```
/cluster/OVA/healthchecks/logs
```

After any routing change, prove that checks still run, per-node evidence is retained, head-node syslog noise stops, failures remain visible, and no core validation is silently skipped.

14. Troubleshooting guide

14.1 Rack status is stale or every validation is pending

1. Check snapshot age and worker status.
2. Confirm PythonCM broker/inventory health.
3. Confirm the bounded log worker completed at least once.
4. Verify configured legacy and deterministic campaign log roots.
5. Compare `/api/status` with the UI.
6. Never replace stale with zero/pass.

14.2 A log cannot be found

1. Capture rack and job ID.
2. Check the activity's recorded stdout path.
3. Check `/cluster/OVA/logs/SU<nn>/rack<nn>/multiNodeNCCL` or `multiNodeNeMo`.
4. Confirm exact basename/job mapping and readable permissions.
5. For full-rack mode, confirm the log itself declares 18 nodes.

14.3 PythonCM command fails

Treat certificate/login/cookie/ProgramRunner transport failure as infrastructure/inconclusive, not hardware failure. Verify the broker, native UUID persistence, status/output path, and permissions. Do not relabel an unexecuted test as failed hardware.

14.4 Slurm fallback is pending

Because the recipe requested an exact allocation, one drained/down node can block the whole job. Inspect the exact requested nodes and reasons. `ReqNodeNotAvail` does not prove general cluster contention.

14.5 IMEX failure

Capture the exact readiness/fabric error and job/type first. Drain only the implicated node with the actual reason, then repair and rerun. MNUberGEMM multi-node cannot pass without active IMEX.

14.6 Multi-rack NCCL hangs in initialization

Separate two known signatures:

- HCOLL failure during `MPI_Init`: disable HCOLL for the accepted recipe.
- NVLS clique collision: all ranks show Init START, none show Init COMPLETE, GPUs idle while ranks spin; use the approved NVLS workaround and escalate the fabric CliqueId configuration.

14.7 UI source change is not visible

- A plain container restart cannot add a new bind mount.
- Backend and SPA assets must be from the same build.
- Run the deployment drift check before overwriting the live deployment tree.
- Recreate through Compose when mounts/image changed, then run the deployment verifier.

- Ask browser users to hard reload after a frontend rebuild.

15. Safety matrix

Class	Examples	Operator requirement
Read-only	Refresh, status, logs, parser results, system info, metrics, API reference	Confirm scope and freshness
Workload-changing	Launch/cancel validation, image staging, workflow execution	Review exact target, provider, image, IDs, and runtime
Node-state-changing	Drain/resume, GPU/IMEX reset, service restart, power/BMC, MAC/NVSwitch fix	Preserve evidence, confirm exact node, coordinate and record action

Never infer node fault from SSH timeout, permission denial, launcher `execve`, missing log, missing telemetry, old non-incrementing counters, or a peer rank's collective-wide error.

16. Beginner handoff checklist

- [ ] Correct active project/cluster and PythonCM provider confirmed
- [ ] Exact rack(s) and node list recorded
- [ ] Preflight/readiness result saved
- [ ] Test type and image/digest/version recorded
- [ ] Outer activity UUID and all native child IDs recorded
- [ ] Expected/received node, rank, collective, and step counts compared
- [ ] First decisive error and source log path preserved
- [ ] Drain/reset/cancel action and exact reason recorded
- [ ] Node Down sheet updated where applicable
- [ ] Rerun result and new IDs recorded
- [ ] Fresh telemetry confirmed after recovery

17. Glossary

Term	Meaning
BCM	Base Command Manager, the cluster-management system
PythonCM	Structured BCM Python API used as the primary inventory/management provider
ProgramRunner	PythonCM command execution with native UUID lifecycle
Fallback	An explicit provider path retained because PythonCM cannot reproduce an accepted recipe's semantics
Outer activity	Durable Rack Status record that may own one or more native child jobs
SU	Super-unit grouping supplied by authoritative PythonCM NodeGroup membership
Logical rack	Compute rack identity derived from real Nscale hostnames
Full rack	Normally 18 GB300 compute trays / 72 GPUs, proven by actual target/result evidence
IMEX	NVIDIA inter-node NVLink fabric service/state required by multi-node NVLink work
NVLS	NVLink SHARP transport behavior used by NCCL where correctly configured
Inconclusive	Infrastructure prevented the test from proving pass or hardware failure
Canonical snapshot	Last complete rack-status snapshot eligible for publication

18. Installation and deployment guide

This part is for the administrator installing or upgrading Rack Status. The commands are templates: resolve every placeholder and inspect the release manifest before execution. The supported NscaleTX model runs Rack Status on the app/observability server and integrates with BCM/PythonCM and bounded Slurm functionality. **The web application must not receive unrestricted root access on the head node.**

18.1 Deployment topology

```
operator browser
      |
      | HTTPS or approved SSH tunnel to :5056
      v
appserver.cm.cluster
  Rack Status container
  /app/backend and /app/bin       read-only source mounts
  /app/var                        named persistent volume
  /cluster/OVA/logs               validation evidence
  /cluster/bringup                read-only bring-up results
  /run/nvis-pythoncm              broker socket mount
      |
      +-- BCM/PythonCM structured inventory and ProgramRunner
      +-- bounded Slurm commands for state/reason and accepted recipes
      +-- VictoriaMetrics/Loki/Alloy observability sources
      +-- exact compute-node allocations and approved container staging
```

Mail delivered to `root@master` does not prove a command ran on the head node. At this site, local root/security mail from the app server is relayed centrally. Always read the originating host in the message body and `Received` chain.

18.2 Installation inputs

Prepare these inputs before starting:

- release source tarball and its SHA-256 value;
- Git bundle or repository checkout at the same release commit;
- Docker Engine with the Compose v2 plugin;
- an app-server account in the Docker group—use `docker`, never `sudo docker`, for Rack Status container health checks;
- a persistent deployment directory with enough space for source and SPA assets;
- a persistent Docker volume for `/app/var`;
- approved BCM client identity, CA/certificate/key or broker cookie files outside Git;
- SSH identity only if the configured execution path actually requires SSH;
- `/cm` and Munge access for the accepted local Slurm fallback;
- read/write validation evidence root and read-only bring-up/container/provenance roots;
- the site-approved login mode and secret supplied through an ignored environment file or secret mount;
- approved validation container images with recorded digest/size.

Minimum host checks:

```
docker version
docker compose version
docker info --format '{{json .SecurityOptions}}'
df -h /cluster/OVA /var/lib/docker
id
test -r /cm/shared/apps/slurm/etc/slurm/slurm.conf
test -d /cluster/bringup
test -d /cluster/OVA/logs
```

Stop on a missing prerequisite. Do not compensate by granting blanket sudo or by copying secrets into the source tree.

18.3 Verify and unpack the release

```
RELEASE=NVIS_Rack_Status_16K_PythonCM_20260914.tar.gz
sha256sum "$RELEASE"
# Compare with the value supplied in the release handoff.

install -d -m 0750 /opt/nvis/rack-status
tar -tzf "$RELEASE" | sed -n '1,80p'
tar -xzf "$RELEASE" -C /opt/nvis/rack-status
cd /opt/nvis/rack-status
```

Before trusting the archive, confirm it does not contain absolute paths, `..` traversal, private keys, live `.env` files, cookies, tokens, runtime databases, caches, or operator logs. Confirm the version and release audit inside the package.

For a Git-bundle handoff:

```
git bundle verify NVIS_Rack_Status_16K_PythonCM_20260914.bundle
git clone NVIS_Rack_Status_16K_PythonCM_20260914.bundle rack-status
cd rack-status
git log -1 --show-signature --oneline
git status --short
```

The archive and Git bundle must represent the same source. Record both hashes in the change ticket.

18.4 Secrets and permissions

Keep runtime secrets outside the checkout. A safe layout is:

```
/opt/nvis/rack-status/             reviewed source, no secrets
/etc/nvis-rack-status/             root/operator controlled environment
/etc/nvis-rack-status/secrets/     mode 0600 identities/tokens
Docker volume rack-data            activities, history, reports, settings
/cluster/OVA/logs/                 validation evidence
```

Required practices:

- `RACK_WEB_AUTH_TOKEN`, BCM credentials, Google token, SSH private key, and cookies are never committed or embedded in the image.
- Credential files are mode `0600`, parent directories `0700` or `0750`, and mounts read-only.
- The application account gets only the groups/ACLs needed for Docker, logs, Munge, and the approved broker socket.
- No browser-originated feature may execute arbitrary shell text.
- Keep sudo `mail_badpass` enabled. Fix an unnecessary `sudo` at its caller; do not suppress the security signal.

18.5 Configure NscaleTX

Copy the packaged example to an ignored runtime file and set site values. The current NscaleTX essentials are:

```
RACK_CLUSTER_ID=nscale-tx
RACK_CLUSTER_NAME=NscaleTX-16K
RACK_CLUSTER_CUSTOMER_NAME=NscaleTX
RACK_CLUSTER_PRODUCT_LINE=GB300
RACK_PLATFORM=bcm
RACK_SCHEDULER=pythoncm
RACK_EXEC_MODE=local
RACK_VALIDATION_CT_RANGE=11-18,28-37
RACK_NATIVE_EXPECTED_RACK_NUMBERS=01-08,17-24
RACK_STATUS_BACKGROUND_INTERVAL_SEC=30
RACK_BRINGUP_HOME=/cluster/bringup
RACK_STATUS_LOG_DIR=/cluster/OVA/logs/nscaletx-16k-validation
RACK_VALIDATION_PROVENANCE_ROOT=/cluster/OVA/scripts/nscaletx-16k-validation
RACK_VALIDATION_CONTAINER_ROOT=/cluster/OVA/containers/nvftp-8k-known-good/20260824
RACK_METRICS_URL=http://127.0.0.1:8428
```

Pin the Nscale Slurm binaries and configuration because non-login shells do not inherit module setup:

```
RACK_SLURM_SKIP_MODULE_LOAD=1
RACK_SLURM_SBATCH=/cm/local/apps/slurm/25.05/bin/sbatch
RACK_SLURM_SRUN=/cm/local/apps/slurm/25.05/bin/srun
RACK_SLURM_SQUEUE=/cm/local/apps/slurm/25.05/bin/squeue
RACK_SLURM_SINFO=/cm/local/apps/slurm/25.05/bin/sinfo
RACK_SLURM_SCONTROL=/cm/local/apps/slurm/25.05/bin/scontrol
RACK_SLURM_SCANCEL=/cm/local/apps/slurm/25.05/bin/scancel
RACK_SLURM_SACCT=/cm/local/apps/slurm/25.05/bin/sacct
RACK_SLURM_CONF=/cm/shared/apps/slurm/etc/slurm/slurm.conf
```

Keep bounded result indexing configured. `RACK_BRINGUP_SCAN_JOBS` and cache TTLs prevent the web refresh path from rescanning an unbounded `/cluster/bringup` history. Do not point a recursive scanner at `/cluster` or systemd-journal.

18.6 BCM/PythonCM integration

PythonCM is version-coupled to BCM. Use the PythonCM library and interpreter shipped with the BCM head; do not replace it with an unrelated PyPI package.

The preferred production design is one boot-enabled broker owning a serialized PythonCM connection and exposing a restricted Unix socket such as:

```
NVIS_PYTHONCM_SOCKET=/run/nvis-pythoncm/query.sock
NVIS_BCM_PYTHON=/cm/local/apps/python312/bin/python3
NVIS_BCM_HOST=127.0.0.1
NVIS_BCM_PORT=8081
```

Bootstrap procedure:

1. Place the service certificate, key, CA, or cookie where only the broker identity can read it.
2. Verify the server certificate fingerprint out of band before enabling any CA-bootstrap option.
3. Install the packaged broker unit from `deploy/nvis-pythoncm-broker.service` on the host that has the matching PythonCM library.
4. Create `/run/nvis-pythoncm` with a group that permits the Rack Status service to access only the broker socket.
5. Enable the broker and confirm its socket, ownership, and startup log.
6. Run the packaged version probe from the same identity and confirm PythonCM path, BCM version, TLS identity, and NodeGroup visibility.
7. Test read-only inventory first. Then use an approved QA node for ProgramRunner submit/status/result/cancel acceptance.

Example administrative commands (paths and account are site-controlled):

```
install -m 0644 deploy/nvis-pythoncm-broker.service /etc/systemd/system/
systemctl daemon-reload
systemctl enable --now nvis-pythoncm-broker.service
systemctl --no-pager --full status nvis-pythoncm-broker.service
stat /run/nvis-pythoncm/query.sock
```

These are installation-time administrator commands, not actions the Rack Status browser performs. Production service-health monitoring belongs to NHC/Alloy or the site's standard platform monitoring, not a background `journalctl` loop.

18.7 Validation scripts and containers

`bin/scripts/` in the release is the canonical script tree. Preview checksum differences before copying:

```
bin/deploy-scripts.sh --dry-run <approved-head-alias>
bin/deploy-scripts.sh <approved-head-alias>
```

The default does not delete remote-only files. Never add `--delete` until those files have been inventoried and approved. After sync, compare checksums for launchers, parser helpers, Slurm initialization includes, and image manifests.

Container preparation is part of deployment, but downloads must be explicit, authenticated, resumable, and verified. The packaged download/staging scripts cover the approved NCCL/NeMo, MNUberGEMM, Llama, DCGM, HPC-X, and NVLoom assets. Record:

- source registry/NVFTP object;
- immutable digest or SHA-256;
- expected byte size;
- local canonical path;
- exact destination nodes;
- per-node receipt and free-space result.

Never launch validation merely because a filename exists. The preflight must verify readability, format, size/digest, and presence on every exact allocated node. A partial or mismatched stage is blocked, not silently repaired by shrinking the allocation.

18.8 Build and install

Run source qualification before building:

```
python -m pytest backend/tests/ -v --tb=short
ruff check backend/ --exclude tests
bash scripts/check_no_vault_in_git.sh

cd frontend
npm ci --ignore-scripts
npx tsc -b --noEmit
npm run lint
npx vitest run
npm run build
cd ..

git diff --check
bin/write-build-info.sh
docker compose build rack-status
```

Do not build the SPA in an unreviewed live tree. Build from the release checkout, then verify that `static/spa/index.html` references assets present in `static/spa/assets/`.

For the current app-server Compose deployment:

```
docker compose config > /tmp/rack-status-compose.resolved.yml
# Review the resolved file without sharing secret values.
docker compose up -d --force-recreate rack-status
docker compose ps
```

Use `--force-recreate` when the image, environment, volumes, or mounts changed. `docker restart` does not apply a new mount or environment variable.

18.9 Persistent state and mounts

The following must survive container recreation:

- durable activity/job registry;
- workflow runs and task output metadata;
- node issues and post-run history;
- generated reports;
- settings/override history where enabled;
- PythonCM job manifests;
- validation source logs and provenance.

Keep `/app/var` on the `rack-data` named volume. Do not copy runtime state into Git or bake it into a new image. The current directory mounts for `/app/backend` and `/app/bin` are read-only; new backend modules are visible because the directory is mounted, but still require a service recreate/restart to be imported by the running Python process.

Back up before upgrade:

```
docker run --rm \
  -v rack-data:/data:ro \
  -v "$PWD/backups":/backup \
  alpine sh -c 'cd /data && tar -czf /backup/rack-data-predeploy.tar.gz .'
sha256sum backups/rack-data-predeploy.tar.gz
```

18.10 Deployment gate

Run the packaged verifier against the actual container name:

```
NVIS_CONTAINER=rack-status-rack-status-1 bin/verify-deploy.sh
```

The gate checks that the container is genuinely running, backend modules import, tree and container modules agree, required durable routes exist, mounted SPA assets match the image, every referenced JS/CSS asset resolves, HTTP responds, and recent boot logs contain no import failure.

Then perform authenticated read-only smoke tests:

1. Login redirects and session creation work.
2. Compute Overview displays the intended cluster and PythonCM badge.
3. Racks loads authoritative SU/rack inventory.
4. Rack Detail displays 18 exact tray nodes and scheduler state/reason.
5. Validation Status displays 14 SUs and the approved 4,032-node campaign denominator.
6. Selecting a floor tile opens evidence without an API error.
7. Logs, Metrics, Events, Analytics, Reports, Projects, Catalog, Tools, API, and Settings load.
8. CSV/PDF/JPEG exports contain the selected domain and do not leak secrets.
9. No browser refresh creates sudo security mail or queries systemd-journal.
10. `docker logs` shows no crash loop, import error, or high-frequency failing request.

Do not include a validation launch in the unattended deployment gate. Hardware execution requires the separately approved QA allocation and change window.

19. First validation acceptance

After the installation gate passes, use one healthy, idle, unreasoned QA node and one approved QA rack. Never use production nodes simply to make the UI green.

1. Confirm exact node identity and current scheduler state/reason.
2. Run the read-only NVLink and image preflight.
3. Launch Hardware Deep Audit on one QA node; retain outer activity UUID and native task ID.
4. Verify expected GPUs, terminal state, output persistence, and no drain on pass.
5. Run NVLink/Network Deep Audit sequentially; verify active links/ports and counter-delta semantics.
6. Run the approved single-rack validation only after full-rack eligibility and staging receipts pass.
7. Confirm the job appears in Scheduler, Logs, parser results, durable history, and Validation Status.
8. Confirm cancellation propagates to every non-terminal child using a disposable QA activity only.
9. Confirm a parser or transport failure is inconclusive and does not automatically drain hardware.
10. Record every ID, digest, exact target, timestamp, and artifact path in the deployment ticket.

Pinned workload contracts:

Workload	Deployment acceptance
NCCL	Approved five-collective recipe, exact expected ranks/nodes, terminal collective evidence, accepted NVLS/MNNVL settings
NeMo	Nemotron-H 56B FP8 image/recipe, expected 30-step acceptance target where configured, terminal throughput/loss evidence
MNUberGEMM	One independent job per rack/NVLink domain, positive-work evidence, no cross-rack job collapse
Llama 405B	Exact approved multi-rack scale, image, model and preflight; fail closed below required campaign size

20. Upgrade, rollback, and recovery

20.1 Upgrade

1. Announce target cluster and maintenance window.
2. Run `bin/prod-drift.sh` against both deployment tree and remote script surface.

3. If exit status is `1`, harvest and review every live change before continuing.
4. Back up `rack-data`, runtime configuration, and ignored deployment overrides.
5. Verify the release bundle/commit and test evidence.
6. Build/stamp from a clean checkout.
7. Dry-run script sync, deploy scripts, recreate Compose, and run the verifier.
8. Run the read-only UI/API checklist.
9. Perform QA validation only when explicitly approved.

20.2 Rollback

Rollback means restoring a coherent set: Git/source, image, SPA index and all hashed assets, environment/mount declarations, canonical script tree, and compatible durable-state schema. Do not restore only `app.py` or only `index.html`.

```
docker compose down
# Restore the previous reviewed checkout/image/configuration.
docker compose up -d --force-recreate rack-status
NVIS_CONTAINER=rack-status-rack-status-1 bin/verify-deploy.sh
```

Restore `rack-data` only when the release notes declare state migration incompatible and the backup matches the application version. Preserve post-upgrade evidence before restoration.

20.3 Common deployment failures

Symptom	Likely cause	Safe response
Blank SPA	`index.html` references missing hashed chunks	Restore a coherent SPA build and run `verify-deploy.sh`
Backend route 404	source/image/mount mismatch or missing import	Recreate with matching backend directory; verify routes/ imports
Every result pending	stale/unreadable evidence root or bounded index has not completed	Check configured roots, cache worker and exact job attribution; never force green
CPU rises during refresh	unbounded log/history scan, repeated failing upstream, or too many active browser refreshes	Inspect request timings and bounded cache configuration; do not add journal polling
Sudo security mail	app code invoked unnecessary `sudo` (for example `sudo docker`)	Call the permitted binary directly; retain sudo notifications
Job stays pending	exact requested node is unavailable or recipe preflight blocked	Read scheduler reason and exact allocation; do not silently shrink
PythonCM unavailable	broker/socket/TLS/library mismatch	Mark provider unavailable, repair broker identity; do not substitute an untracked shell path
Metrics empty	upstream, labels, WAL, or freshness failure	Prove endpoint and current series; empty is not zero

21. Security and operational boundaries

- Rack Status has no business scanning `journalctl` as the interactive app-server account.
- NHC and the approved Alloy monitoring stack own general system-service monitoring.
- Read-only head-node observability must be deliberately scoped and reviewed; it is not a justification for unrestricted sudo.
- Validation launchers may use Slurm/Pyxis/MPI because those semantics are intrinsic to the accepted test, not because the UI needs general head-node access.
- The head-node and app-server responsibilities remain distinct. Mail relaying, shared filesystems, and network paths do not erase the execution origin.

- All mutating API calls require authentication, exact target validation, audit IDs, and explicit UI confirmation.
- No touchless installer should manufacture credentials, disable host security controls, accept unknown TLS identities, or select production nodes for validation.

22. Release evidence checklist

- [ ] Source archive hash and Git bundle hash recorded
- [ ] Git commit, version, build stamp, and dirty state recorded
- [ ] Secret scan and archive-content review passed
- [ ] Backend and frontend suites passed at release-recorded counts
- [ ] Shell syntax and static checks passed
- [ ] Container image ID/digest recorded
- [ ] Persistent-volume backup hash recorded
- [ ] Drift checks passed or every change harvested
- [ ] Canonical scripts synchronized by checksum
- [ ] Deployment verifier passed
- [ ] All screenshots/routes reviewed
- [ ] Validation Status denominator/domain/export verified
- [ ] Rack Detail scheduler state/reason verified
- [ ] No `sudo docker`, journal polling, or security-mail regression
- [ ] Approved QA acceptance IDs and results attached
- [ ] Rollback version and operator named

23. Screenshot index

Figure/screen	File	Purpose
Cluster Overview	`01-cluster-overview.png`	cluster/provider orientation
Racks	`02-racks.png`	SU/rack status and filters
Rack Detail	`03-rack-detail-validation.png`	exact nodes and validation controls
Slurm state/reason	`03b-rack-node-slurm-reason.png`	per-node scheduler context
Multirack	`04-multirack.png`	exact campaign selection
Scheduler	`05-scheduler.png`	activities and provider-native IDs
Logs	`06-validation-logs.png`	retained validation evidence
Metrics	`07-metrics.png`	observability and honest errors
Validation floor	`08-validation-status-floor.png`	14-SU/4,032-node status
Node evidence	`08b-validation-node-evidence.png`	Job ID/reason/provenance dialog
Analytics	`09-analytics-results.png`	structured result analysis
Events	`10-events.png`	scheduler reasons and activity events
Reports	`11-reports.png`	durable handoff artifacts
Projects	`12-projects-clusters.png`	active cluster identity
Catalog	`13-platform-catalog.png`	hardware/platform definitions
Tools	`14-tool-registry.png`	validation tool contracts
API	`15-api-reference.png`	running route reference
Settings	`16-settings.png`	runtime configuration visibility

24. Provenance and current references

This guide was checked against:

- the complete 68-page `NVIS_Rack_Status_Guide_v1.1.pdf` source;
- the live deployed NscaleTX 16K UI on 14 September 2026;
- current `README.md`, `INSTALL.md`, and `docs/ENVIRONMENT.md`;
- `docs/RACKSTATUS_USER_GUIDE.md` and `docs/NSCALE_TX_DEPLOYMENT.md`;
- `docs/PYTHONCM_MIGRATION.md` and the PythonCM hybrid audit;
- `docs/VALIDATION_STATUS_IMPLEMENTATION_2026-09-02.md`;
- current Compose, deploy, drift, script-sync, build-stamp, and deployment-verification code;
- the live frontend route registry and API reference.

Screenshots are timestamped release evidence, not permanent health claims. The deployed application, durable activity records, source logs, current controller/PythonCM state, and approved campaign manifest remain authoritative.

END OF COMBINED USER AND DEPLOYMENT GUIDE · VERSION 3.0